



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Практическое занятие № 1/4ч.

Разработка мобильных компонент анализа безопасности информационно-аналитических систем

Уровень	(наименование дисциплины (модуля) в соответствии с учебным планом)
Форма обучения	специалитет (бакалавриат, магистратура, специалитет)
Направление(-я) подготовки	очная (очная, очно-заочная, заочная)
Институт	10.05.04 «Информационно-аналитические системы безопасности» (код(-ы) и наименование(-я))
Кафедра	кибербезопасности и цифровых технологий (полное и краткое наименование)
Используются в данной редакции с учебного года	КБ-2 «Информационно-аналитические системы кибербезопасности» (полное и краткое наименование кафедры, реализующей дисциплину (модуль))
Проверено и согласовано « ____ » ____ 20 ____ г.	2024/25 (учебный год цифрами)
	(подпись директора Института/Филиала с расшифровкой)

Москва 2025 г.

ОГЛАВЛЕНИЕ

ГЛАВА 1. НАСТРОЙКА СРЕДЫ РАЗРАБОТКИ ANDROID STUDIO.	3
ГЛАВА 2. ВВЕДЕНИЕ В СОЗДАНИЕ ПРИЛОЖЕНИЙ	10
2.1 Описание структуры проекта	10
2.2 Создание приложения	11
2.3 Описание интерфейса среды разработки	14
2.4 Создание эмулятора.....	18
2.5 Запуск приложения.....	21
ГЛАВА 3. КОМПОНЕНТЫ ЭКРАНА И ИХ СВОЙСТВА.	22
ГЛАВА 4. ВИДЫ LAYOUT. КЛЮЧЕВЫЕ ОТЛИЧИЯ И СВОЙСТВА.....	27
4.1 Контрольное задание.....	28
4.1.1 LinearLayout	28
4.1.2 TableLayout.....	29
4.1.3 ConstraintLayout	30
4.2 Задание.....	31
ГЛАВА 5. LAYOUT-ФАЙЛ В ACTIVITY. СМЕНА ОРИЕНТАЦИИ ЭКРАНА ...	33
5.1 Layout-файл	33
5.2 Задание:.....	33
5.3 Ориентация экрана	34
5.4 Задание.....	37
ГЛАВА 6. ОБРАЩЕНИЕ ИЗ КОДА К ЭЛЕМЕНТАМ ЭКРАНА. ОБРАБОТЧИКИ СОБЫТИЙ.	38
ГЛАВА 7. ОБРАБОТЧИКИ СОБЫТИЙ	40

ГЛАВА 1. НАСТРОЙКА СРЕДЫ РАЗРАБОТКИ ANDROID STUDIO.

Android Studio является официально поддерживаемой Google IDE для разработки Android-приложений. Основанная на IntelliJ IDEA, Android Studio доступна под лицензией Apache 2.0. Google предоставляет Android Studio для Windows, Mac OS X и Linux. Загрузить данное программное обеспечение возможно с официальной страницы приложения: <https://developer.android.com/studio>. Перед установкой Android Studio требуется убедиться, что операционная система и компьютер соответствует следующим требованиям:

ОС Windows:

- Microsoft Windows 8/10/11 (64-разрядная версия);
- 8 ГБ оперативной памяти и более;
- 8 ГБ свободного места на диске (IDE + Android SDK + Android Emulator);
- Минимальное разрешение экрана 1280 на 800 пикселей.

Mac OS:

- MacOS 10.14 (Mojave) или выше;
- 8 ГБ оперативной памяти и более;
- 8 ГБ свободного места на диске (IDE + Android SDK + Android Emulator);
- Минимальное разрешение экрана 1280 на 800 пикселей.

ОС Linux:

- любой 64-битный дистрибутив Linux с поддержкой Gnome, KDE или Unity DE;
- x86_64 CPU архитектура, процессор Intel Core второго поколения или новее, или процессор AMD с поддержкой виртуализации AMD (AMD-V) и SSSE3;
- библиотека GNU C (glibc): версия 2.31 или более поздняя;
- 8 ГБ оперативной памяти;
- 8 ГБ свободного места на диске;
- минимальное разрешение экрана 1280 на 800 пикселей;

Для повышения производительности работы эмулятора мобильного устройства необходимо: процессор Intel с поддержкой функций Intel VT-x, Intel

EM64T (Intel 64) и Execute Disable (XD) или процессор AMD с поддержкой технологии виртуализации AMD (AMD-V).

Далее требуется произвести запуск среды разработки (рис.1.1):

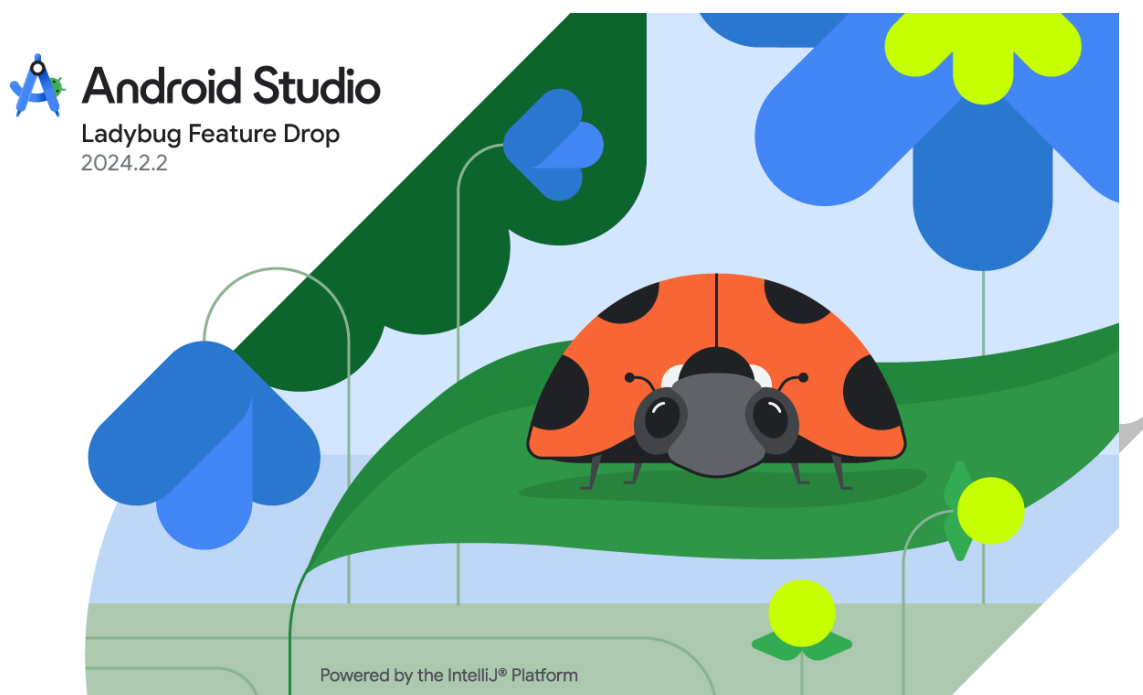


Рисунок 1.1 - Окно приветствия Android Studio

При первом запуске предлагается установить в диалоговых окнах несколько параметров конфигурации (рис.1.2). В первом диалоговом окне основное внимание уделяется установке настроек из ранее установленной версии *Android Studio*.

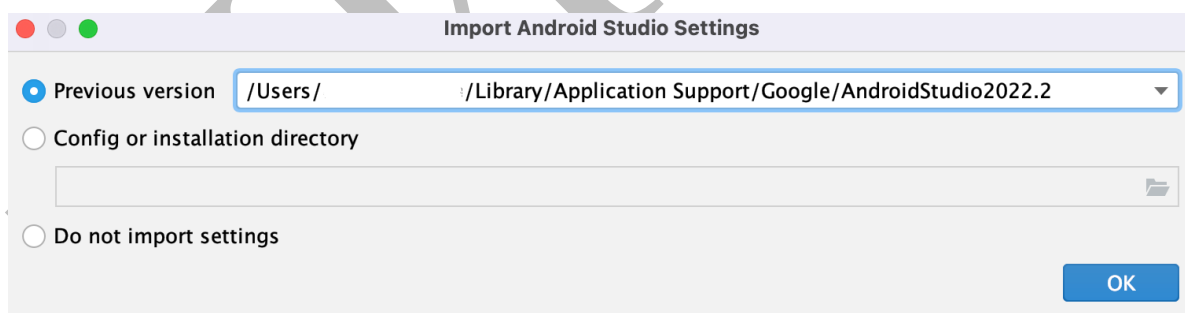


Рисунок 1.2 – Параметры импорта настроек

Возможно принять настройки по умолчанию и нажать на кнопку «ОК». После этого *Android Studio* выведет диалоговое окно «Мастера установки» (рис.1.3):

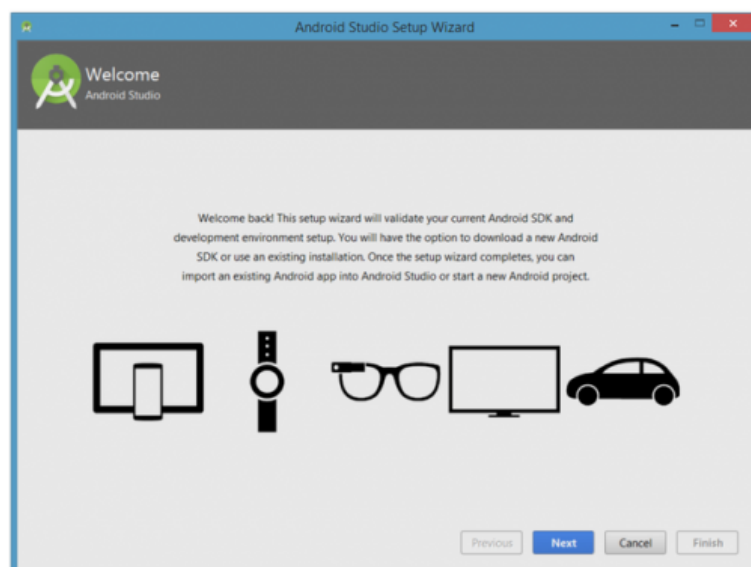


Рисунок 1.3 – Статус настроек Android SDK и среды разработки

После нажатия кнопки «Далее», «Мастер установки» предложит выбрать тип установки компонентов SDK (рис.1.4). Рекомендуется использовать стандартную конфигурацию. Стоит обратить внимание на выбранные версии SDK платформ – в рамках курса разработки, требуется, чтобы были установлены SDK для Android 15.0 (*VanillaIceCream*) и Android 8.0 (*Oreo*).

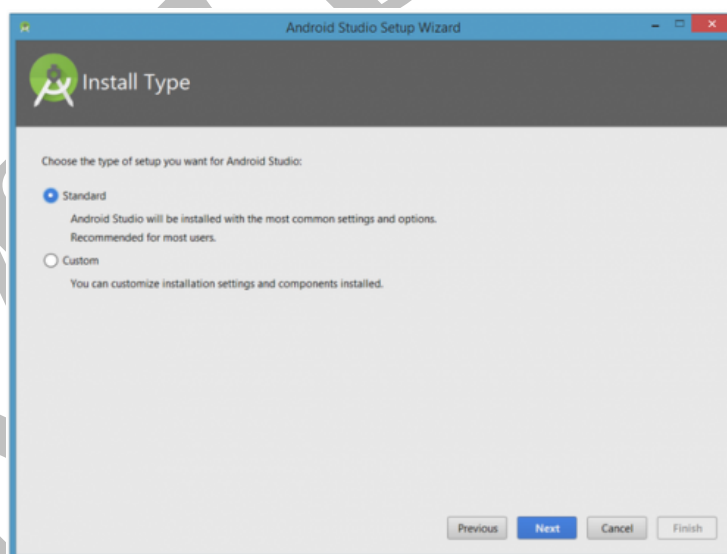


Рисунок 1.4 – Экран выбора типа установки

Далее требуется подтверждение выбранных настроек (рис.1.5).

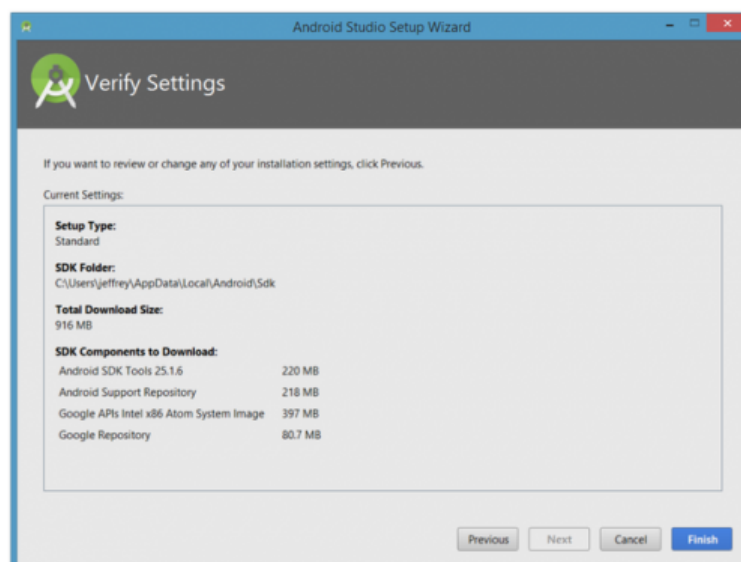


Рисунок 1.5 – Экран настроек среды разработки

«Мастер установки» произведёт загрузку и распаковку необходимых компонентов (рис.1.6). Кнопка «Показать детали» предназначена для отображения подробной информации о загружаемых архивах и их содержанием:

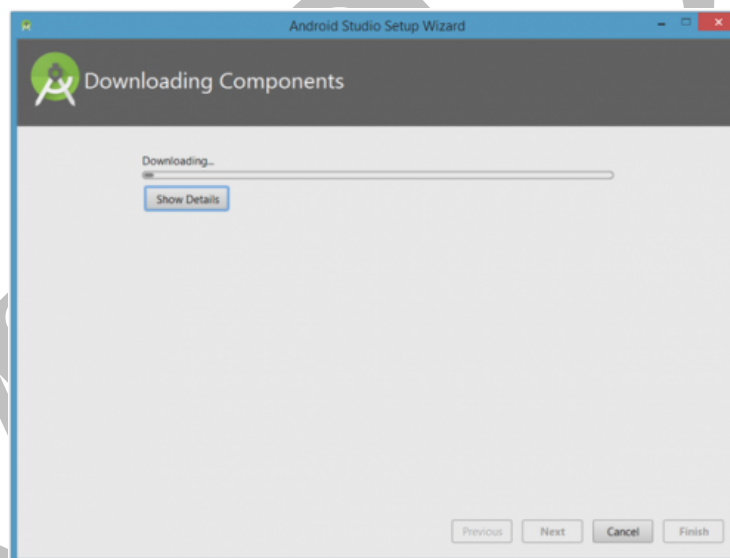


Рисунок 1.6 – Экран загрузки и установки компонентов среды разработки

После нажатия кнопки «Готово» производится завершение работы «Мастера установки» и отображение диалогового окна «Welcome to Android Studio» (рис.1.7):

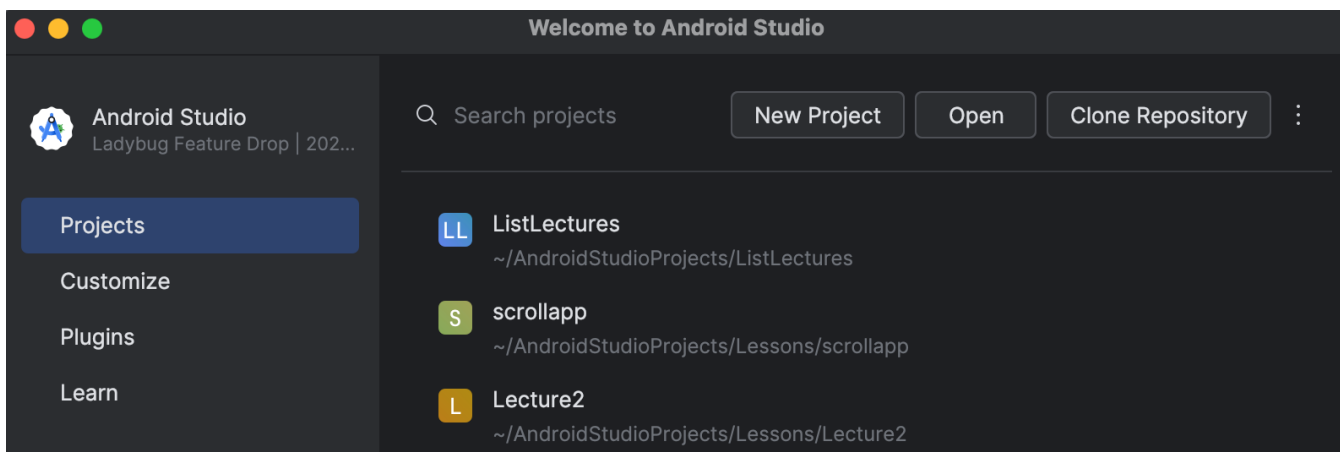


Рисунок 1.7 – Экран приветствия среды разработки

В случае, если компьютер не поддерживает аппаратную виртуализацию, возможно использовать существующие аналоги эмуляторов устройств (напр. Genymotion) или реальное *Android*-устройство.

Также возможен переход в SDK-менеджер из главного меню. Необходимо вызвать меню в верхнем правом углу и нажать SDK Manager (рисунок 1.8).

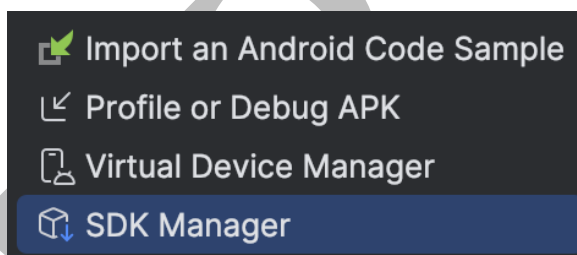


Рисунок 1.8 – Меню настроек

Далее необходимо отметить необходимые версии Android SDK и нажать Apply (рисунок 1.9).

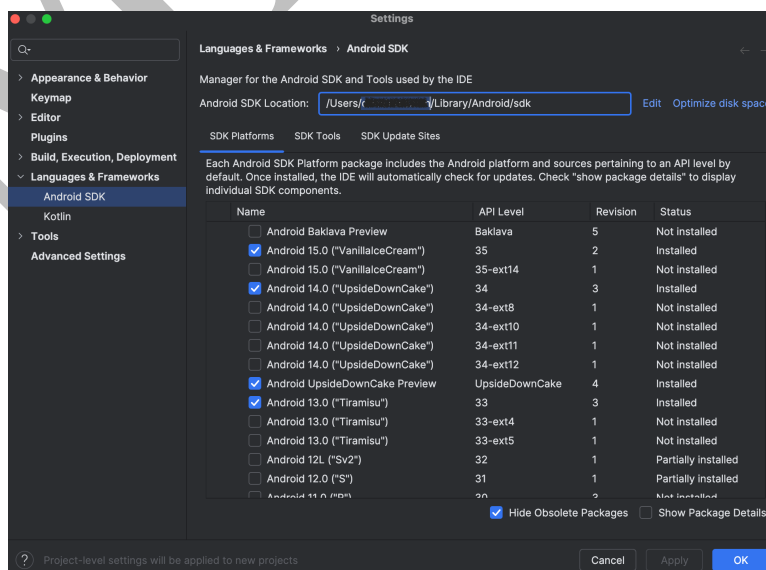


Рисунок 1.9 – Экран приветствия среды разработки

Во вкладке SDK Tools возможно установить различный инструментарий обеспечения процесса разработки мобильных приложений (рисунок 1.10).

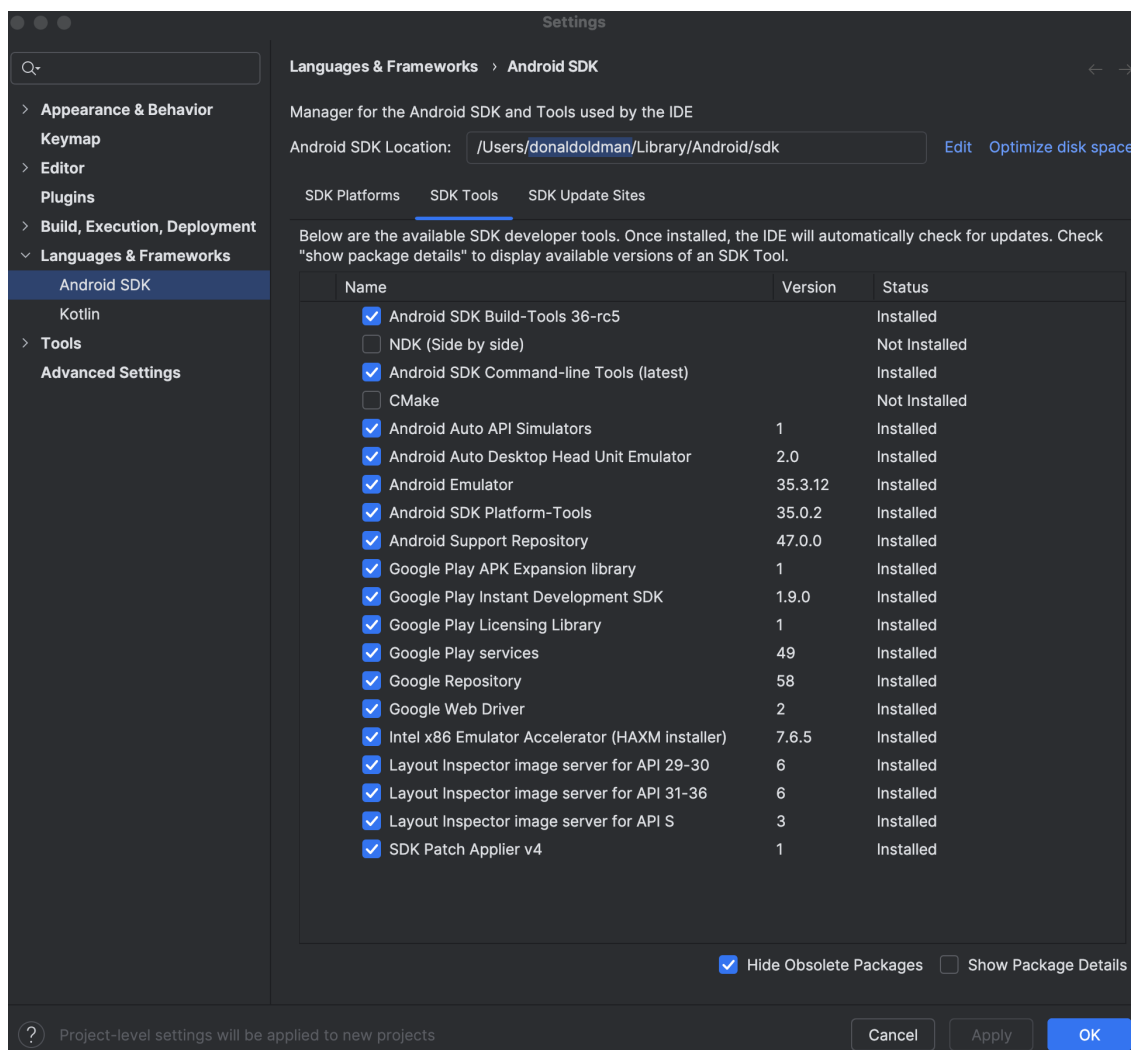


Рисунок 1.10 – Экран настроек. Вкладка SDK Tools

Функциональные особенности и инструменты среды разработки:

- единая среда, в которой возможно разрабатывать приложения для всех типов Android-устройств;
- возможность создания приложений под Android TV, Android Automotive OS (для функционирования на приборных панелях автомобилей) и Android Wear;
- использование шаблонов для создания проектов и компонентов *Android*;
- функциональный редактор макетов, позволяющий перемещать компоненты пользовательского интерфейса и включающий в себя возможность предварительного просмотра макетов на нескольких экранах;

- автоматизированный рефакторинг кода и быстрые исправления;
- поддержка системы автоматической сборки «*Gradle*»;
- инструменты «*Lint*» для повышения производительности и устранения проблем, связанных с совместимостью версий и т.д.;
- интеграция с инструментом оптимизации и обфускации *Java*-кода «*ProGuard*»;
- создание цифровой подписи разработчика и автоматизированная сборка приложения;
- многофункциональный эмулятор;
- расширенное поведение для команд запуска и отладки «*Instant Run*» для внесения изменений в запущенное приложение без создания нового файла приложения;
- встроенная поддержка облачной платформы *Google* для интеграции с *Google Cloud Messaging*, *Firebase* и т.д.;
- поддержка *Android NDK* (native development kit), позволяющий реализовывать часть приложения на языках C/C++;
- возможность проведения трассировки приложения;
- перехват сетевого трафика изучаемого приложения;
- наличие установленного по умолчанию *JDK 17*;
- возможность предпросмотра файлов, содержащих пользовательский интерфейс, без запуска на устройстве приложения;
- возможность установки различных дополнительных плагинов.

Разработку приложений возможно производить на языке *Java* или *Kotlin*. Т.к. язык *Kotlin* не изучается в рамках программы института, использоваться будет язык *Java*. Для успешного прохождения данного курса требуются начальные знания *Java*!

ГЛАВА 2. ВВЕДЕНИЕ В СОЗДАНИЕ ПРИЛОЖЕНИЙ

Запуск и тестирование приложения требует наличия эмулятора или реального устройства на базе ОС *Android*. Стандартный эмулятор, поставляемый вместе с *Android* SDK, называется «*Android Virtual Device*» (AVD), позволяет выполнять установку разрабатываемых приложений, производить их запуск и отладку.

2.1 Описание структуры проекта

Для успешной сдачи практических заданий необходимо выполнять требования по размещению кодовой базы приложений. В первую очередь необходимо создать проект, внутри которого автоматически создаётся модуль. В данном модуле производится работа с кодом и внешним видом приложения. Результатом компиляции и сборки модуля является приложение. Таким образом, модуль является приложением, а проект контейнером для модуля/ей.

Структура проекта с одним модулем изображена на рисунке 2.1. Как правило, один проект является одним *Android*-приложением (один модуль).



Рисунок 2.1 – Структура проекта с одним модулем

В одном проекте возможно существование нескольких модулей (рис. 2.2), а также имеется возможность создания нескольких проектов в одном рабочем окружении.



Рисунок 2.2 – Структура проекта с несколькими модулями

На рисунке 2.2 первый проект содержит два модуля, а второй – три. При запуске какого-либо проекта необходимо указать, какой именно модуль требуется запустить.

При выполнении практических занятий требуется создавать один проект на одно занятие, а внутри использовать модули для каждого задания.

2.2 Создание приложения

Для создания проекта требуется открыть окно приветствия Android Studio (рисунок 2.3):

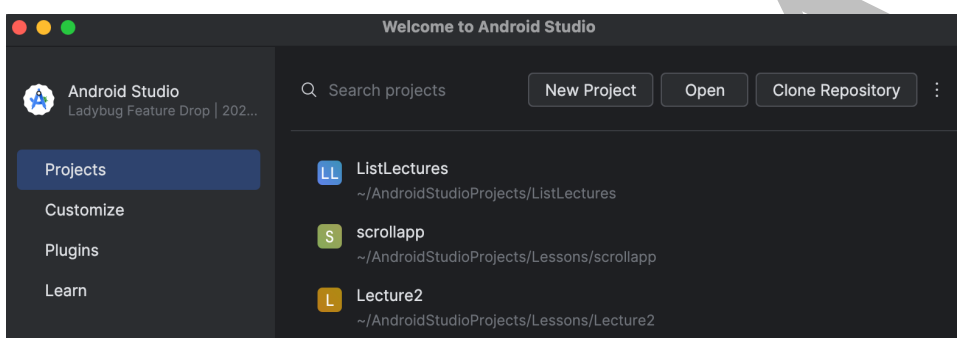


Рисунок 2.3 – Окно приветствия Android Studio

После этого требуется активировать кнопку «New Project». На рисунке 2.4 приведен экран выбора макета приложения, представляющие собой набор шаблонов для разного типа проектов. Некоторые из данных макетов будут рассмотрены в дальнейших занятиях. В рамках текущего занятия необходимо выбрать шаблон «*Empty Views Activity*».

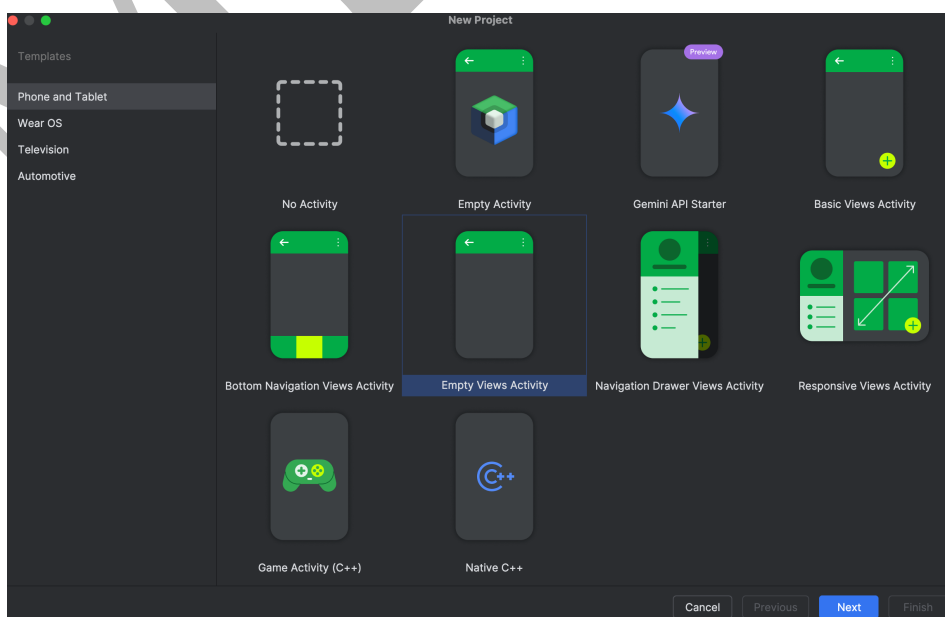


Рисунок 2.4 – Экран выбора макета приложения

После выбора требуемого шаблона требуется определить параметры проекта, приведенные на рисунке 2.5.

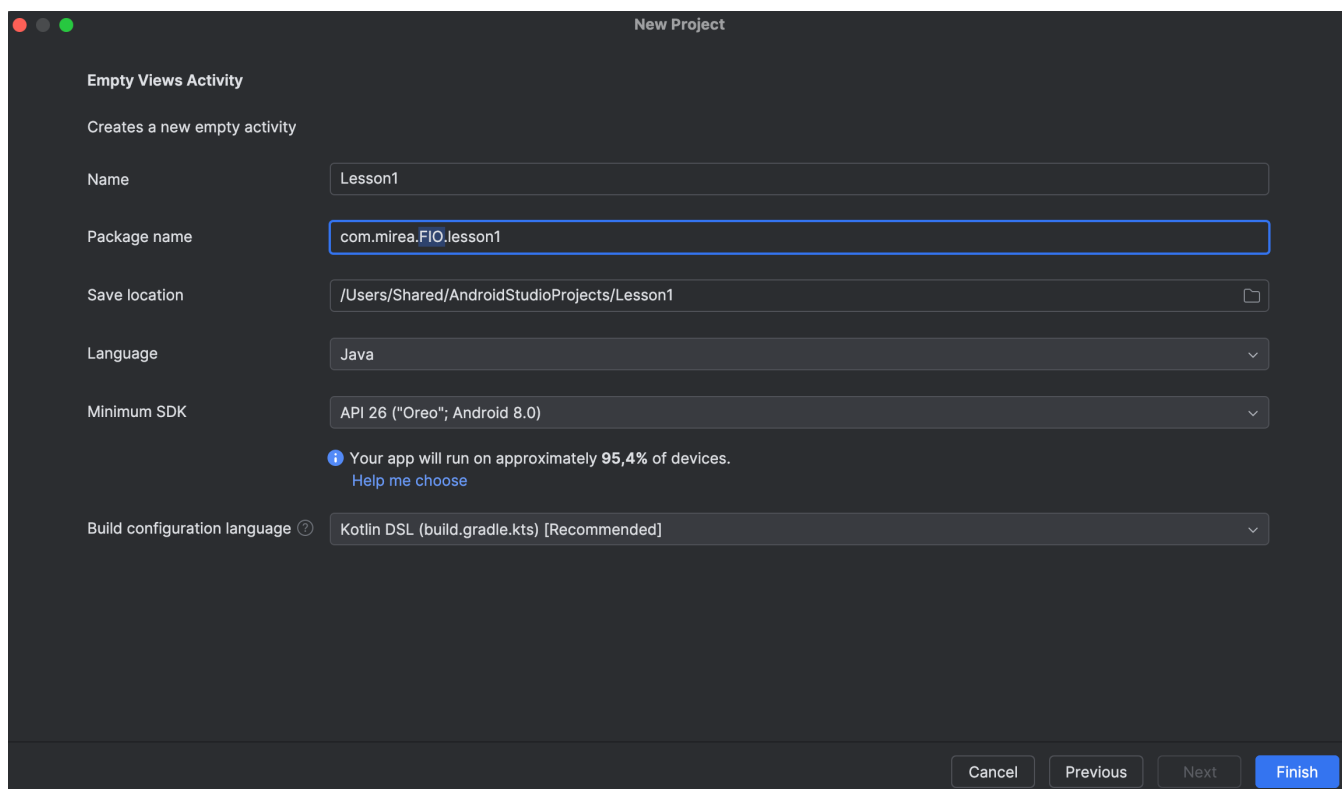


Рисунок 2.5 – Экран конфигурации проекта

К параметрам проекта относятся:

1. «*Name*» – имя проекта, отображаемое в списке проектов при открытии *Android Studio* – «*Lesson1*».
2. «*Package name*» – полное название проекта (согласно правилам именования пакетов в *Java*). Имя пакета должно быть уникальным среди всех пакетов, установленных в ОС *Android*.

В рамках учебной дисциплины требуется соответствие следующей форме: «ru.mirea.ФамилияИО. lessonX».

3. «*Save location*» – директория, в которой будут размещаться файлы проекта.
4. «*Language*» – используемый язык программирования в проекте.
5. «*Minimum SDK*» указывает минимально поддерживаемую версию *Android API*. Ниже информация о том, какой процент приложений ее поддерживает.

В рамках учебной дисциплины требуется использовать минимальное API 26.

6. Кнопка «*Finish*» предназначена для создания проекта.

Требуется дождаться пока произойдет скачивание зависимостей проекта и его сборка. Случай успешной компиляции и сборки проекта приведен на рисунке 2.6.

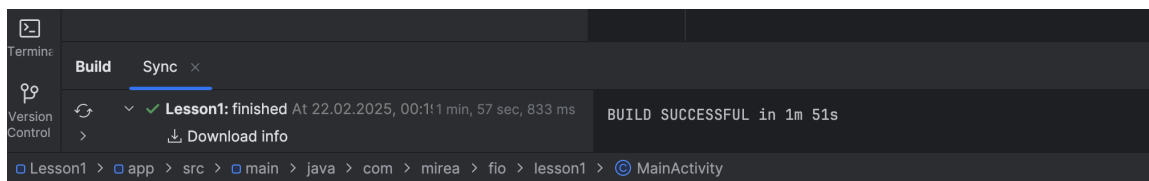


Рисунок 2.6 – Статус сборки проекта

В качестве примера, требуется создать в проекте новый модуль. Процедура частично похожа на создание проекта.

В рамках одного практического занятия требуется создания нескольких модулей.

Для создания модуля требуется в меню выбрать «*File -> New -> New module*». На рисунке 2.7 приведены возможные характеристики создаваемого модуля.

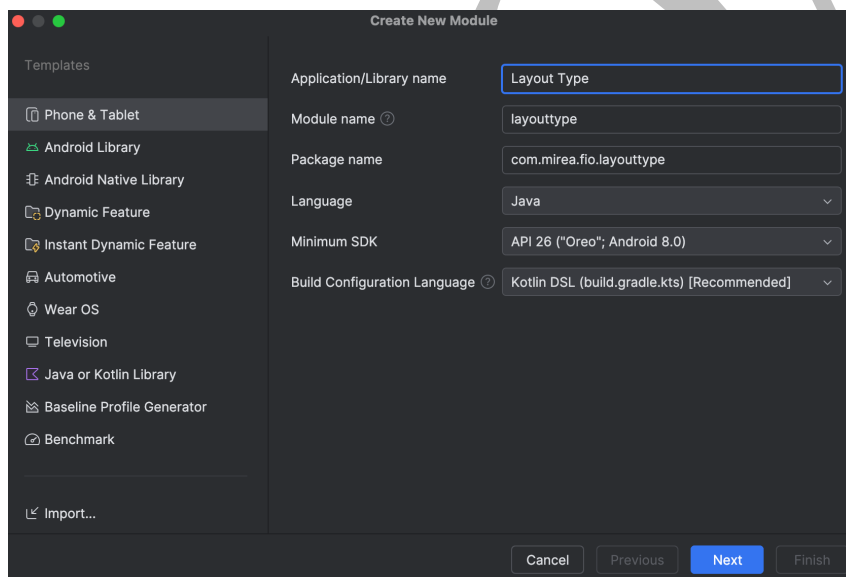


Рисунок 2.7 – Экран создания нового модуля

К параметрам модуля относятся:

1. «*Application/Library name*» – имя приложения, которое будет отображаться в списке приложений в мобильном устройстве. Требуется указать: «*Layout Type*».
2. «*Module name*» – название модуля, отображаемое в структуре проекта.
3. «*Package name*» – имя пакета, содержащее ФИО студента.
4. «*Minimum SDK*» – минимально поддерживаемую версию Android API.

После перехода на следующий экран требуется выбрать тип шаблона проекта «*Empty Views Activity*». В последующих экранах имеется возможность изменения

имен файлов «*activity*» и разметки. Далее все действия производятся аналогично созданию проекта.

2.3 Описание интерфейса среды разработки

В процессе создания нового модуля или проекта в *Android Studio* автоматически генерируется множество файлов и папок. В левой панели *Android Studio* размещена структура проекта (рисунок 2.8). В среде разработки проект возможно представить двумя основными способами: «*Android*» (обобщенный), «*Project*» (подробный). На рисунке 2.9 представлены два типа отображения структуры приложения.:

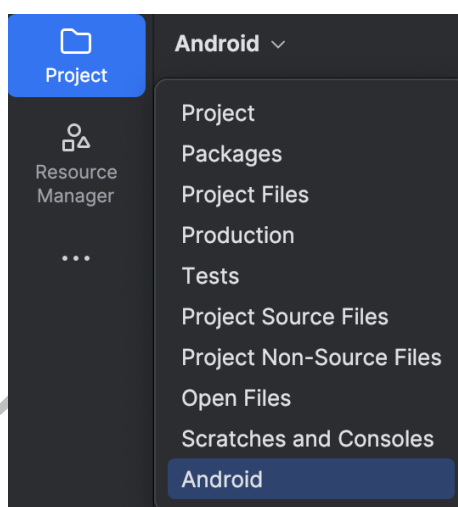
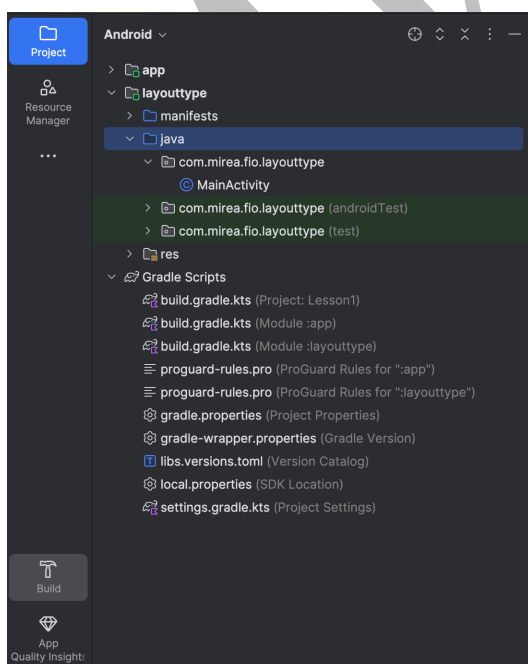
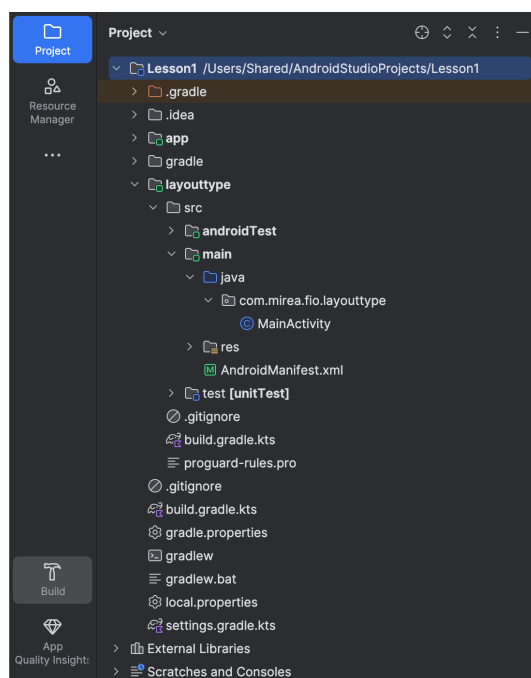


Рисунок 2.8 – Меню проекта



(a)



(б)

Рисунок 2.9 – Структура проекта в (a) – «*Android View*», (б) – «*Project View*»

Файл «*AndroidManifest.xml*» является важной частью приложения, поскольку определяет структуру и метаданные проекта, его компоненты и требования. Данный файл содержит описание всех компонентов проекта и требуемые разрешения, определяет, взаимодействие их друг с другом и другими приложениями.

Директория с исходным кодом *src* разделена на две части: кодовая база приложения и код для тестов. Папка «*res/values*» используется для хранения значений ресурсов цвета, стилей, размеров, значений отступов, значений полей и т.д. В созданном модуле, как правило, размещаются 4 основных ресурсных файла: «*colors.xml*», «*strings.xml*», «*themes.xml*».

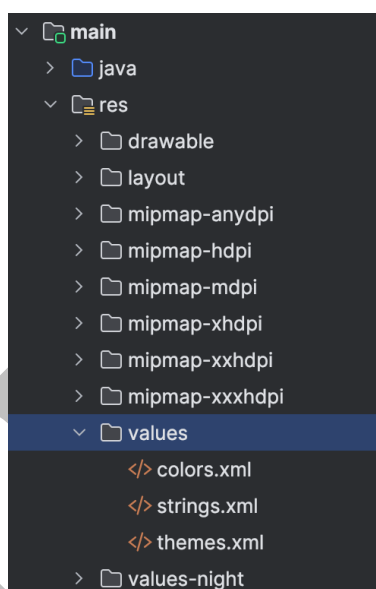


Рисунок 2.10 – Структура директории «*res*»

Для хранения файлов типа *mp3*, *mp4*, *sfb* и т. д. в директории требуется создать папку «*raw*» и разместить в ней файлы. Директория «*assets*» предназначена для размещения файлов типа XML, HTML, шрифты и т.д.

В Android Studio используется система сборки «*Gradle*», которая предназначена для автоматизации процесса сборки, тестирования и развертывания. В дереве проекта находятся несколько файлов с расширением «*.gradle.kts*». Файл с меткой «*Project*» отвечает за весь проект, остальные – за отдельные модули. В приложении может быть несколько модулей и в таком случае у каждого из них будет свой «*.gradle*» файл. Далее рассмотрен файл «*build.gradle*», отвечающий за сборку модуля:

```

plugins {
    alias(libs.plugins.android.application)
}

android {
    namespace = "com.mirea.fio.layouttype"
    compileSdk = 35

    defaultConfig {
        applicationId = "com.mirea.fio.layouttype"
        minSdk = 26
        targetSdk = 35
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
}

dependencies {
    implementation(libs.appcompat)
    implementation(libs.material)
    implementation(libs.activity)
    implementation(libs.constraintlayout)
    testImplementation(libs.junit)
    androidTestImplementation(libs.ext.junit)
    androidTestImplementation(libs.espresso.core)
}

```

Подключаемый плагин «*libs.plugins.android.application*» говорит о том, что модуль является приложением, что требует от системы «gradle» определенного порядка сборки проекта. Если разрабатываемый модуль является библиотекой, то требуется установить значение «*libs.plugins.android.library*». Блок «*android*» содержит следующие поля:

- «*namespace*» – пространство имен приложения, используемое в основном для доступа к ресурсам приложения;
- «*applicationId*» – имя пакета приложения. Должно быть уникальным во всей экосистеме. Если ранее было использовано данное имя пакета, то не будет возможности загрузить это приложение в *Google Play*, или установить на устройство, на котором уже установлено приложение с таким же именем пакета.
- «*compileSdk*» указывает уровень *Android API*, который «*Gradle*» должен

использовать для компиляции приложения. Это означает, что в приложении разработчик имеет возможность использовать функции API, включенные в данный уровень API и ниже;

- «*minSdk*» – определяет минимальный уровень API, необходимый для запуска приложения на мобильном устройстве или эмуляторе;
- «*targetSdk*» – указывает уровень API, используемый для тестирования приложения;
- «*versionCode*» – целочисленное обозначение версии приложения. При выгрузке обновления, например, в *Google Play*, должна обязательно инкрементироваться;

Блок «*buildTypes*» предназначен для настройки возможных типов сборки. По умолчанию система сборки определяет тип сборки «*release*». Тип сборки «*release*» применяет настройки *ProGuard* и по умолчанию не использует ключ разработчика для подписи приложения. Также существует тип сборки «*debug*» явно не указанный в конфигурации сборки, но он включает в себя средства отладки и подписывает приложение отладочным ключом.

Далее расположен блок «*dependencies*», в котором перечислены библиотеки, от которых зависит приложение. В данном блоке перечисляются на отдельных строках зависимости от библиотек с версиями. При обновлении проекта, «*gradle*» производит загрузку всех перечисленных зависимостей. Так же возможно использование «Jar-файлов», которые требуется разместить в директории «*libs*».

Система сборки Gradle включает в себя два файла надстроек, расположенных в корневом каталоге проекта, которые возможно использовать для указания параметров самого набора инструментов «Gradle build toolkit»:

- *gradle.properties* – содержатся параметры проекта, такие как максимальный размер кучи демона Gradle и т.д.;
- *local.properties* – свойства локальной среды для плагина Android Gradle, а именно путь к SDK, CMake и NDK.

2.4 Создание эмулятора

Для тестирования приложения возможно использование эмулятора Android устройства, встроенного в Android Studio. Требуется перейти в настройки эмулятора (рис.2.11).

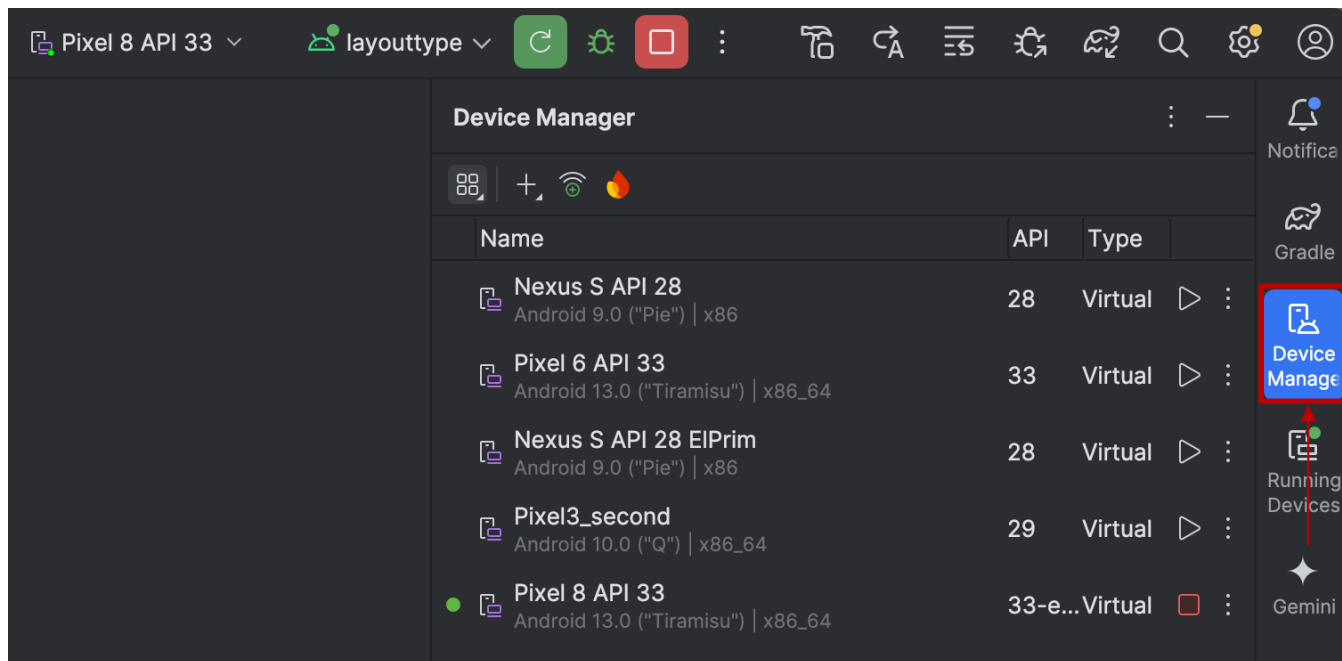


Рисунок 2.11 – Панель управления Android Studio

На рисунке 2.12 приведён экран менеджера виртуальных устройств, в котором отображаются созданные ранее виртуальные устройства.

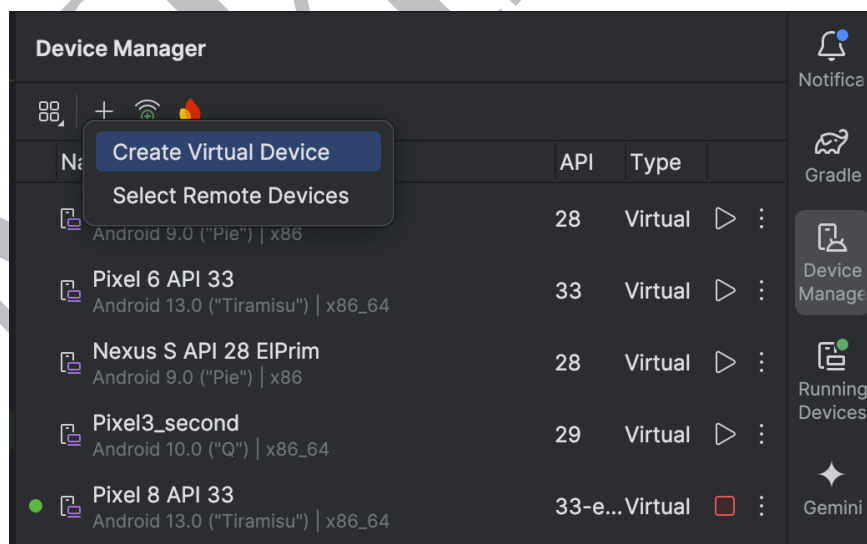


Рисунок 2.12 – Экран менеджера виртуальных устройств

После нажатия кнопки «Create device» отобразится экран, приведенный на рис.2.13.

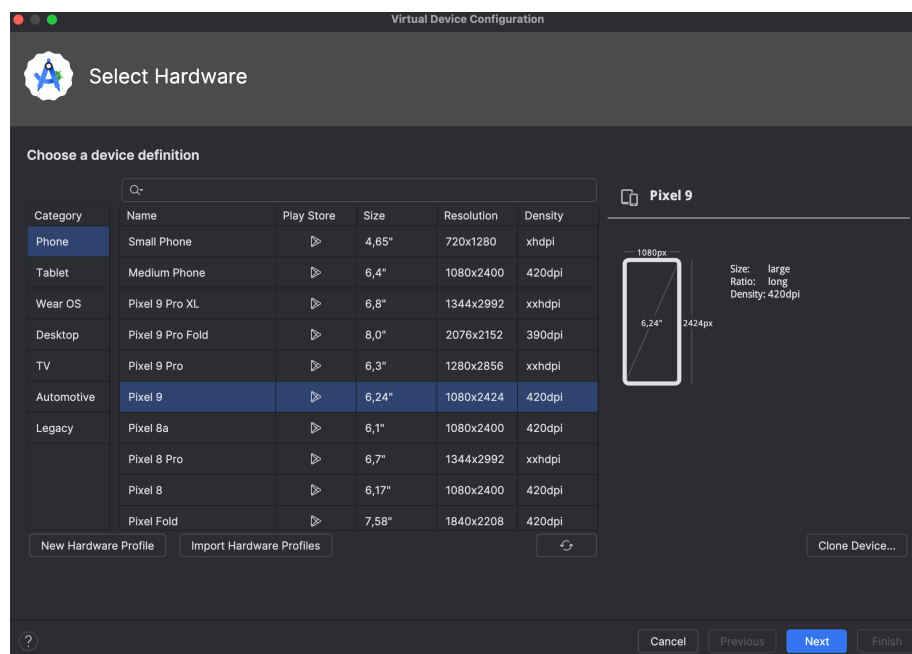


Рисунок 2.13 – Экран настройки виртуального устройства

На представленном экране производится создание устройства Pixel 9. Далее требуется скачать образ операционной системы «*VanillaIceCream*» (рисунок 2.14). Средний размер образа ОС Android составляет 1,6 GB.

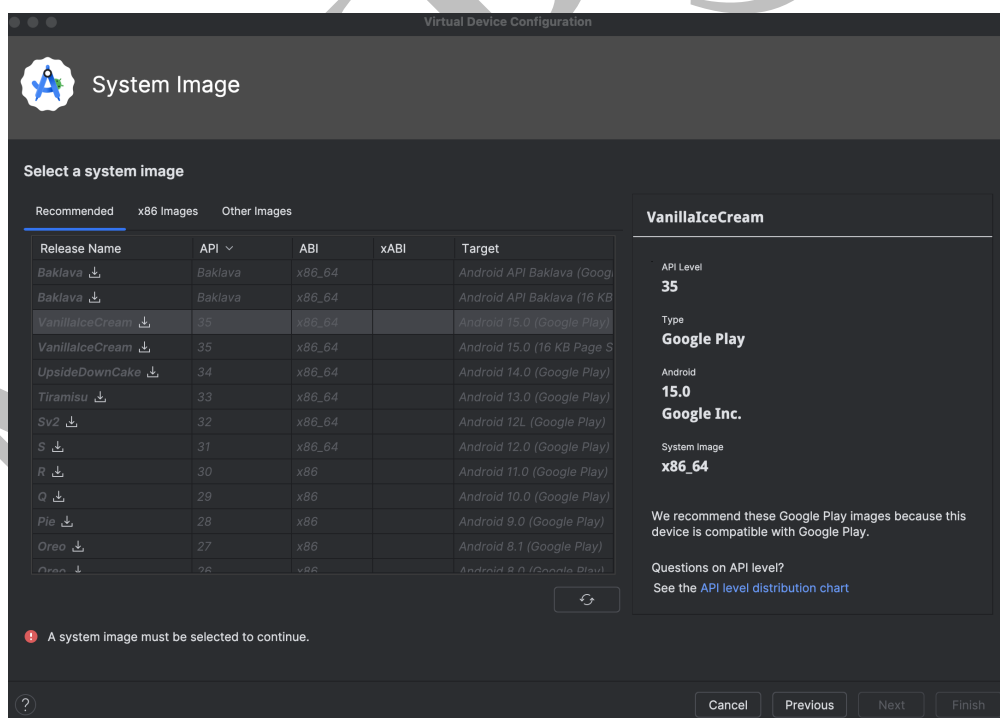


Рисунок 2.14 – Выбор образа системы

Далее указывается название эмулятора с возможностью изменения конфигурации устройства (рисунок 2.15).

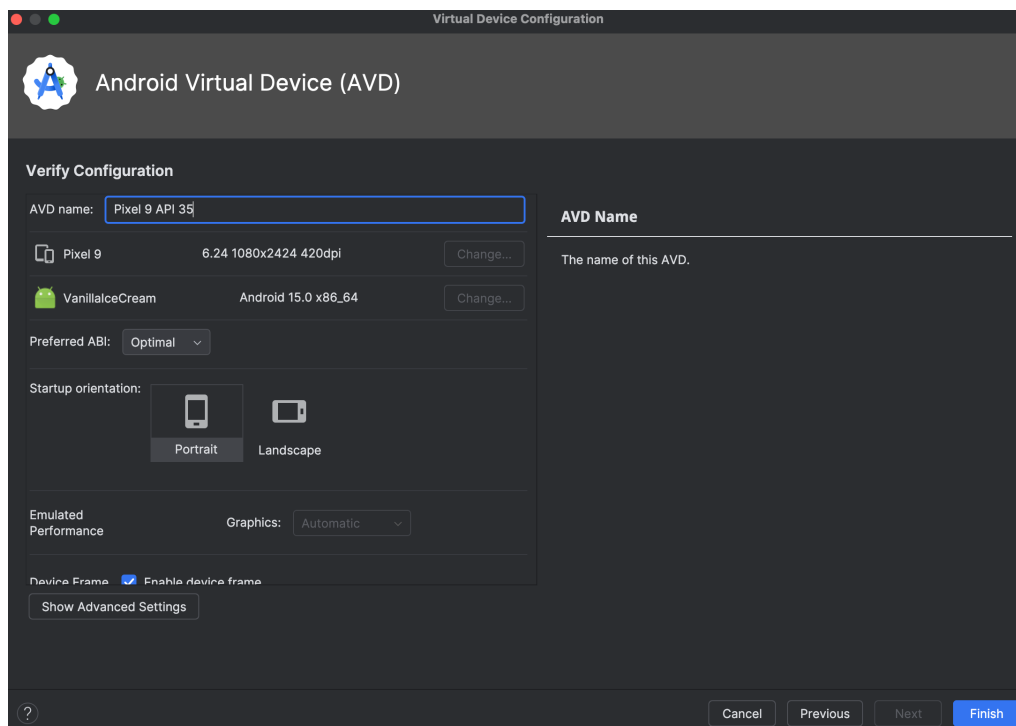


Рисунок 2.15 – Экран конфигурирования эмулятора

После создания эмулятора в менеджере виртуальных устройств отобразится созданное устройство, которое требуется запустить (рисунок 2.16).



Рисунок 2.16 – Внешний вид эмулятора Pixel 9

2.5 Запуск приложения

После успешной компиляции и сборки проекта, имеется возможность запуска приложения на созданном эмуляторе. Требуется выполнить следующие шаги:

- выбрать соответствующий модуль в выпадающем списке: layouttype;
- выбрать запущенный эмулятор (либо без него, тогда отобразится экран менеджера устройств): Pixel 9 API 35;
- нажать кнопку *Run*



Рисунок 2.17 - Панель управления Android Studio

После успешной сборки, приложение должно отобразиться на экране эмулятора (рис.2.17).

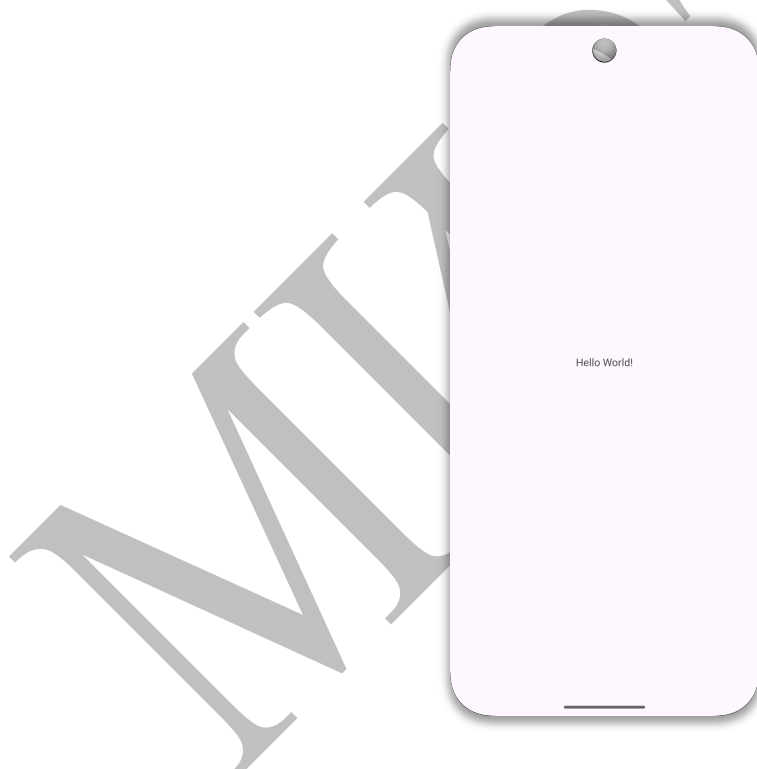


Рисунок 2.18 – Главный экран приложения Layout Type

ГЛАВА 3. КОМПОНЕНТЫ ЭКРАНА И ИХ СВОЙСТВА

Приложения для ОС Android состоит из набора различных экранов. На первом этапе изучения мобильной разработки будут рассмотрены «*activity*». На запущенном эмуляторе отображается одно «*activity*» приложения «*Layout Type*». Как правило одно приложение занимает экран, а пользователь переключается между приложениями с соответствующими экранами. Стоит отметить, что современные версии ОС Android поддерживают возможность деления экрана на несколько частей, что позволяет в один момент времени отображать несколько «*activity*». В качестве наиболее простого примера далее рассмотрено почтовое приложение, содержащее несколько «*activity*» – список писем, просмотр письма и настройки почтового ящика. При работе с приложением производится перемещение по экранам.

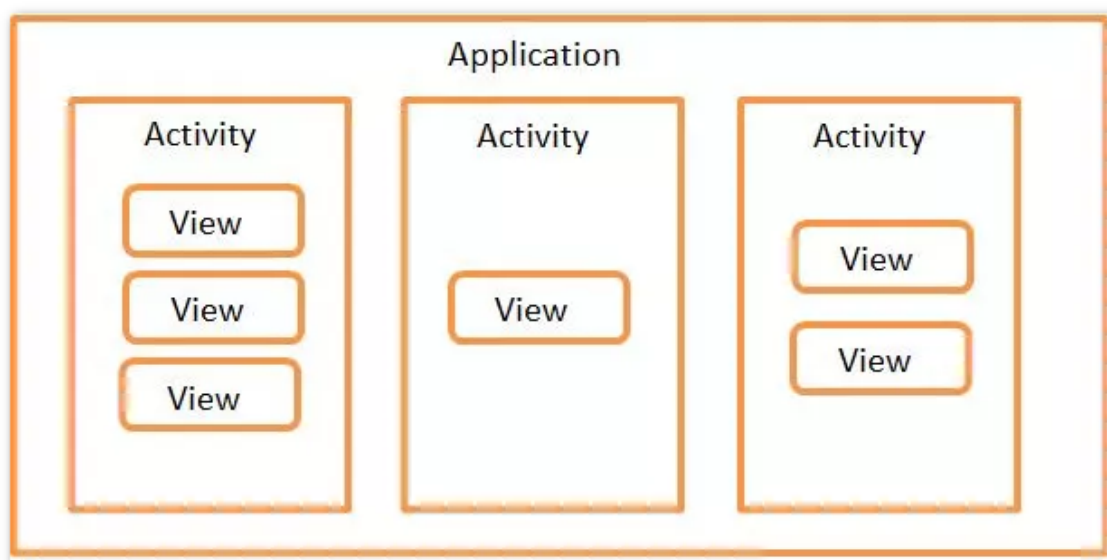


Рисунок 3.1 – Структура View элементов

Содержимое «*activity*» формируется из различных компонентов, называемых «*view*». Самые распространенные «*view*» – это кнопка, поле ввода, чекбокс и т.д. При разработки пользовательских интерфейсов возможно использовать специальные контейнеры «*ViewGroup*». Самый распространенный пример «*ViewGroup*» – это «*layout*», отвечающий за расположение дочерние элементов на экране (таблицей, строкой, столбцом и т.д.).

В режиме отображения структуры проекта «Android» в модуле «*layouttype*» требуется открыть файл «*activity_main.xml*», находящийся в папке «*res>layout*». В данном файле определяется набор и расположение «*view*» компонентов, отображаемых на экране.

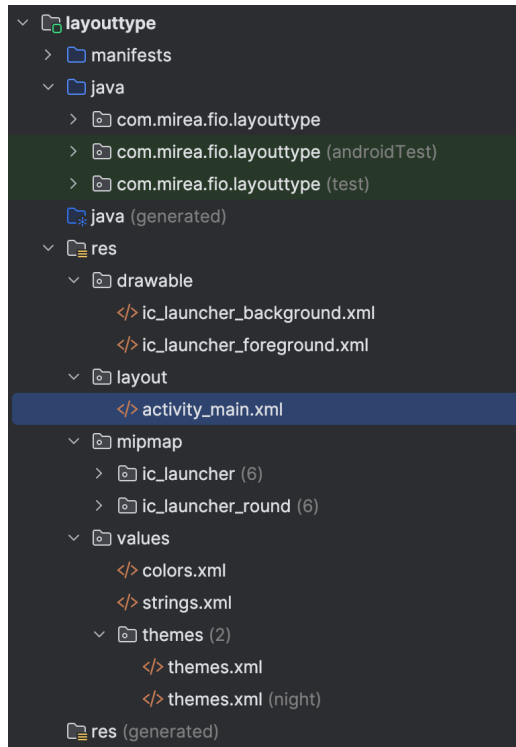


Рисунок 3.2 – Расположение layout

Кроме основной компоновки экрана, существуют дочерние элементы компоновки для группы элементов. Компоновка – это некий визуальный шаблон пользовательского интерфейса приложения, позволяющий воздействовать на элементы управления, их свойства и расположение. Связывание Java-кода и элементов разметки достигается с помощью присваивания элементам уникальных идентификаторов с помощью выражение «*@+id/your_value*» атрибут «*android:id*».

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:id="@+id/main"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">

    <TextView
        android:id="@+id/textViewHello"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

После установки идентификатора UI-компонента возможно обращаться к элементу через код при помощи метода «*findViewById(r.id.your_value)*».

На рисунке 3.3 представлено окно редактирования экранов разметки. Изменения макетов возможно осуществлять с помощью изменений в xml-файле во вкладке «*Code*» или с помощью графического редактора «*Design*».

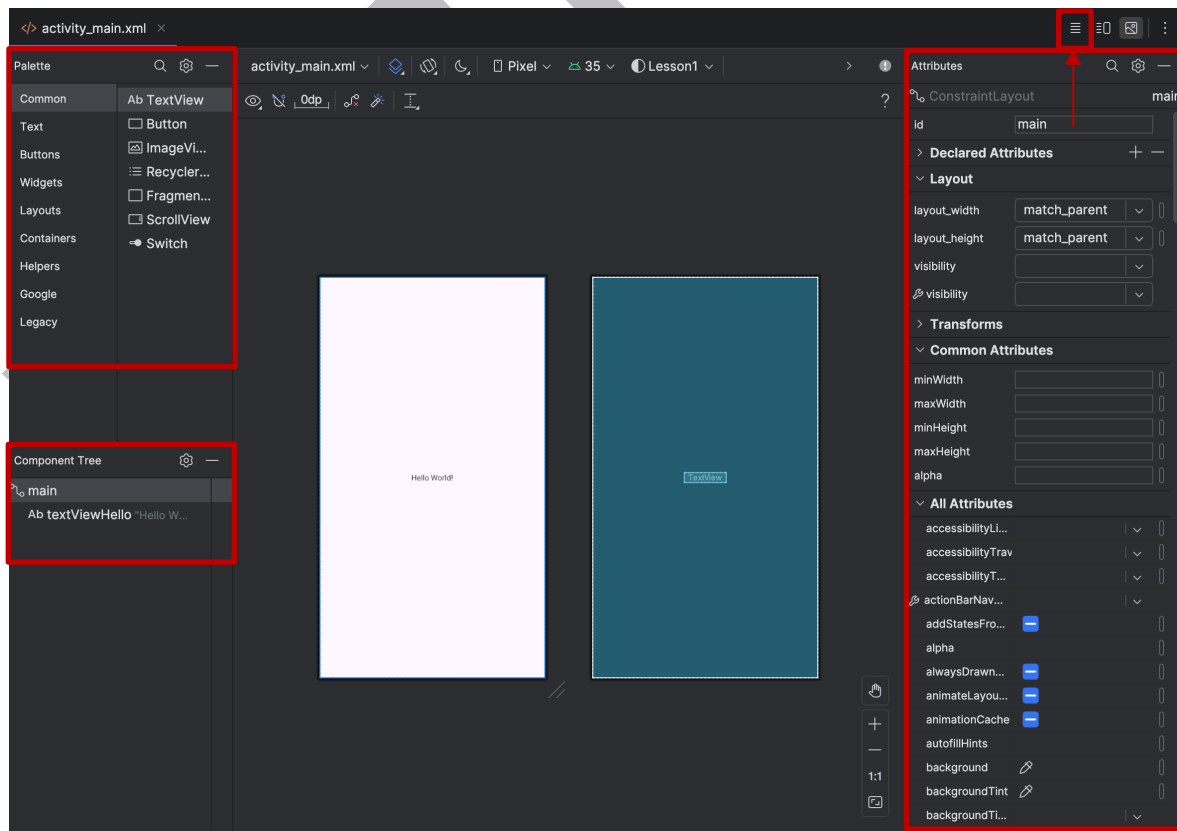


Рисунок 3.3 – Экран редактирования разметки

Экран разметки отображается в двух режимах:

- «*Design*» – отображается «*View*» компоненты так, как они выглядят на экране;
- «*Blueprint*» – отображаются только контуры «*View*» компонентов.

В панели «*Palette*» размещаются список стандартных «*View*» компонентов, которые возможно добавить на экран: кнопки, поля ввода и т.д.

Панель «*Component Tree*» предназначена для отображения иерархии «*View*» компонентов экрана. В представленном примере на рисунке 3.3 корневым элементом является «*ConstraintLayout*», а его дочерним элементом «*TextView*».

Панель «*Attributes*» отображает свойства выбранного компонента. С помощью изменения свойств элементов имеется возможность настраивать внешний вид, расположение и содержимое «*View*» компонента.

Самостоятельно добавить элемент `TextView` на экран и изменить отображаемый текст

Каждый объект «*View*» и «*ViewGroup*» обладает уникальными атрибутами. Некоторые атрибуты характерны только объектам «*View*» (например, объект «*TextView*» и все дочерние классы поддерживают атрибут «*textSize*»). Некоторые атрибуты являются общими для всех объектов «*View*», поскольку они наследуются от корневого класса «*View*» (такие как атрибут «*id*»). Любые другие атрибуты рассматриваются как «параметры макета» и предназначены для описания определенных ориентаций элементов «*View*».

```
<TextView
    android:id="@+id/textView2"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="TextView"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
```

Основными атрибуты графического элемента `textView2` из примера выше являются:

- ключевое слово «*android*» в названии каждого атрибута обозначает принадлежность к элементам «*android*»;
- «*id*» – это идентификатор, используемый для обозначения уникальности

всех объектов «*View*» в иерархии. Во время компиляции приложения идентификатору присваивается целое число, однако до компиляции его тип является строковым. Символ «@» в начале строки указывает на то, что обработчику «xml» требуется произвести синтаксический анализ остальной части идентификатора и определить ее в качестве ресурса идентификатора. Символ «+» является маркером нового ресурса, который необходимо добавить к ресурсам.

– «*layout_width*» (ширина элемента) и «*layout_height*» (высота элемента) могут задаваться в абсолютных значениях с указанием количества размера или с помощью тегов *match_parent* (максимально возможная ширина или высота в пределах родительского контейнера) и *wrap_content* (ширина или высота определяется по содержимому элемента).

Обратите внимание, что у «*View*» -элемента может не быть *id* (*android:id*). Например, для *TextView* он обычно не нужен, т.к. элементы чаще всего статичны, и к ним редко обращаются при работе приложения. При работе с *EditText* – работа производится с содержимым текстового поля, *button* – требует обработки нажатия.

Также в проекте возможно использовать различные медиафайлы. Для добавления изображений возможно использовать два различных подхода:

– копирование изображений в папку «*res/drawable*». Стоит учитывать, что файл изображения будет увеличивать размер выходного образа приложения. Кроме того, работа с большими форматами изображений отрицательно влияют на производительность, поэтому следует использовать небольшие и оптимизированные (сжатые) графические файлы, в том числе векторные.

– вызов контекстного меню у директории «*res*» и выбор элемента «*New>Image/Vector Asset*». Изображение будет сохранено в папке «*/res/drawable*».

ГЛАВА 4. ВИДЫ LAYOUT. КЛЮЧЕВЫЕ ОТЛИЧИЯ И СВОЙСТВА

Размещение на экране графических компонентов (кнопки, поля ввода, чекбоксы и т.п.) производится с помощью использования специальных контейнеров. Основное различие заключается в упорядочивании компонентов внутри контейнера. К основным типам макетов «*ViewGroup*» относятся (рис.4.1):

- «*LinearLayout*» – отображение «*View*»-элементов в виде одной строки (тип «*Horizontal*») или одного столбца (тип «*Vertical*»).
- «*TableLayout*» – отображение элементов в виде таблицы, по строкам и столбцам.
- «*RelativeLayout*» – для каждого элемента настраивается его положение относительно других элементов.
- «*FrameLayout*» – все дочерние элементы прикрепляются к верхнему левому углу экрана. В разметке нельзя определить различное местоположение для дочернего объекта. Последующие дочерние объекты «*View*» отображаются поверх предыдущих компонентов, частично или полностью затеняя их.
- «*ConstraintLayout*» – размещение элементов «*View*» путем привязки к дочерним элементам или самому себе.

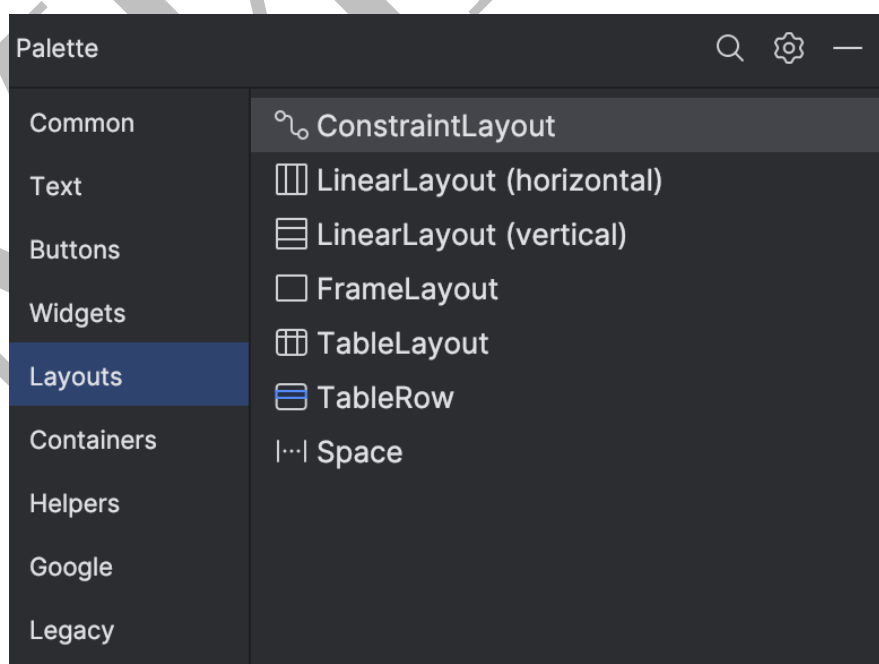


Рисунок 4.1 – Панель выбора контейнеров компоновки

4.1 Контрольное задание

Изучить атрибуты макетов «ViewGroup» и для каждого типа создать свой Layout

4.1.1 LinearLayout

Требуется выделить папку «res/layout» и создать новый «Layout resource file» с именем «linear_layout.xml» (рис.4.2).

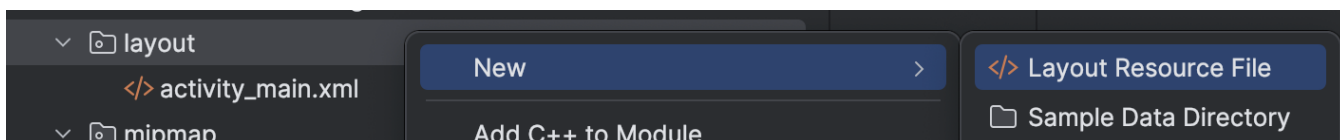


Рисунок 4.2 – Создание макета

Экран создания layout-файлов представлена на рисунке 4.3. В «Root element» требуется указать «LinearLayout».

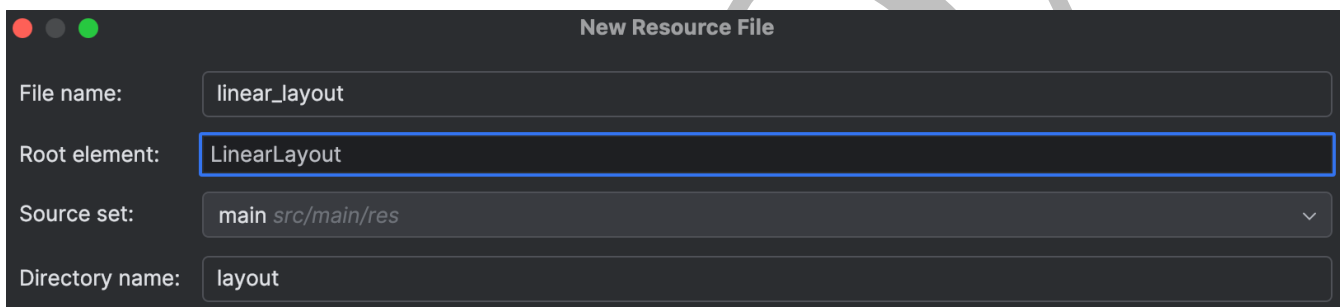


Рисунок 4.3 – Создание макета

Созданный компонент обладает свойством «Orientation», которое определяет, как образом будут расположены дочерние элементы – горизонтальной или вертикальной линией.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

</LinearLayout>
```

Элементы «GroupView» возможно вкладывать друг в друга. В качестве примера, требуется вложить в один «LinearLayout» два других и разместить внутри них элементы «button». Значение ориентации корневого «LinearLayout» устанавливается вертикальной, а двух вложенных макетов – горизонтальной. Стоит отметить, что при создании интерфейса в графическом редакторе требуется перетаскивать элементы на конкретный элемент.

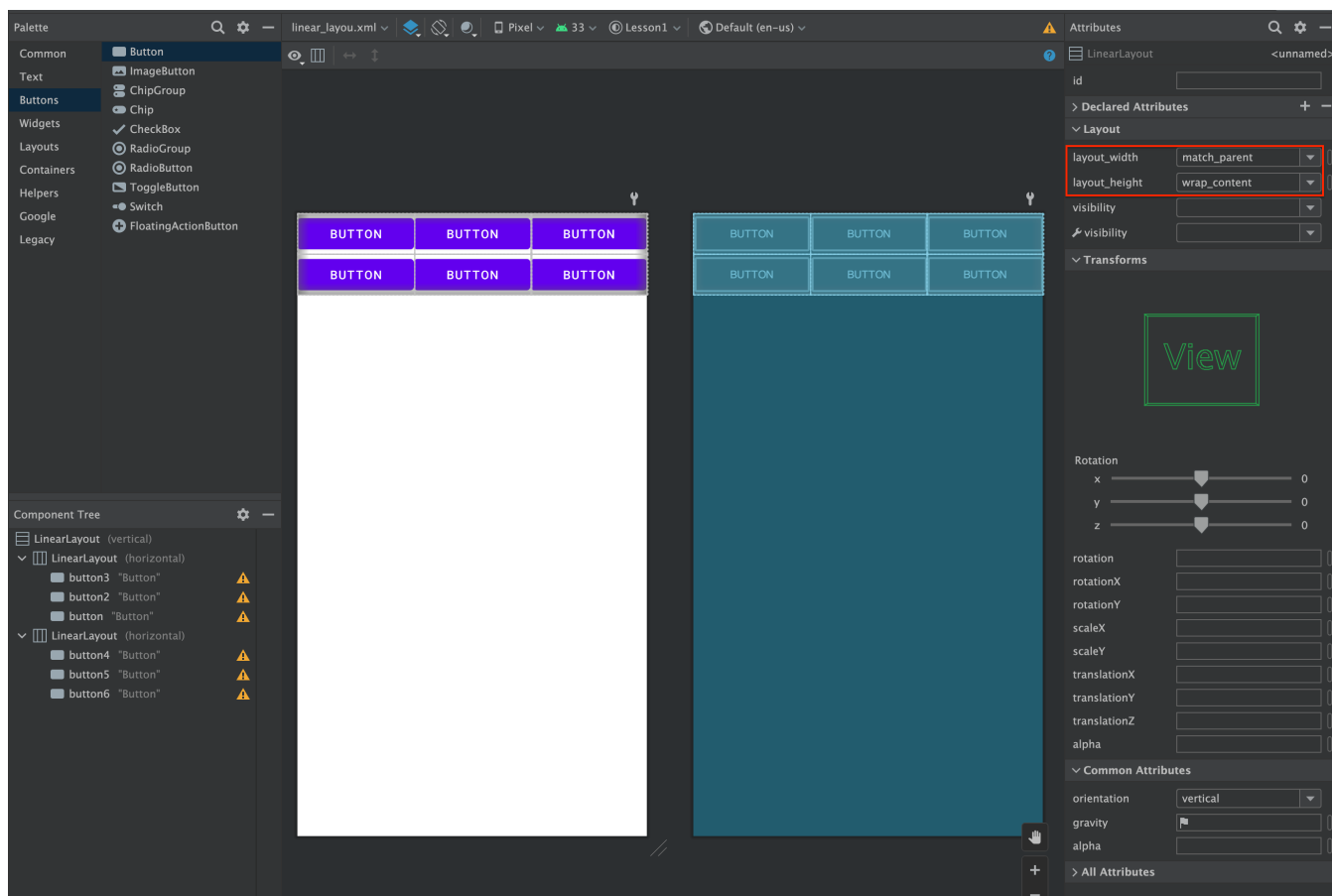


Рисунок 4.4 – Внешний вид экрана приложения LinearLayout

Создайте экран приведённый на рис.4.4

4.1.2 TableLayout

Макет «*TableLayout*» состоит из строк «*TableRow*», каждая в свою очередь может содержать «*View*»-элементы, формирующие столбцы. Т.е. количество элементов в строке — это количество столбцов. Требуется создать файл разметки «*table_layout.xml*» с корневым элементом «*TableLayout*» и добавить в корневой элемент три «*TableRow*»-строки. Стоит отметить, что у каждого вложенного элемента имеется атрибут «*layout_weight*», назначающий весовой коэффициент представления.

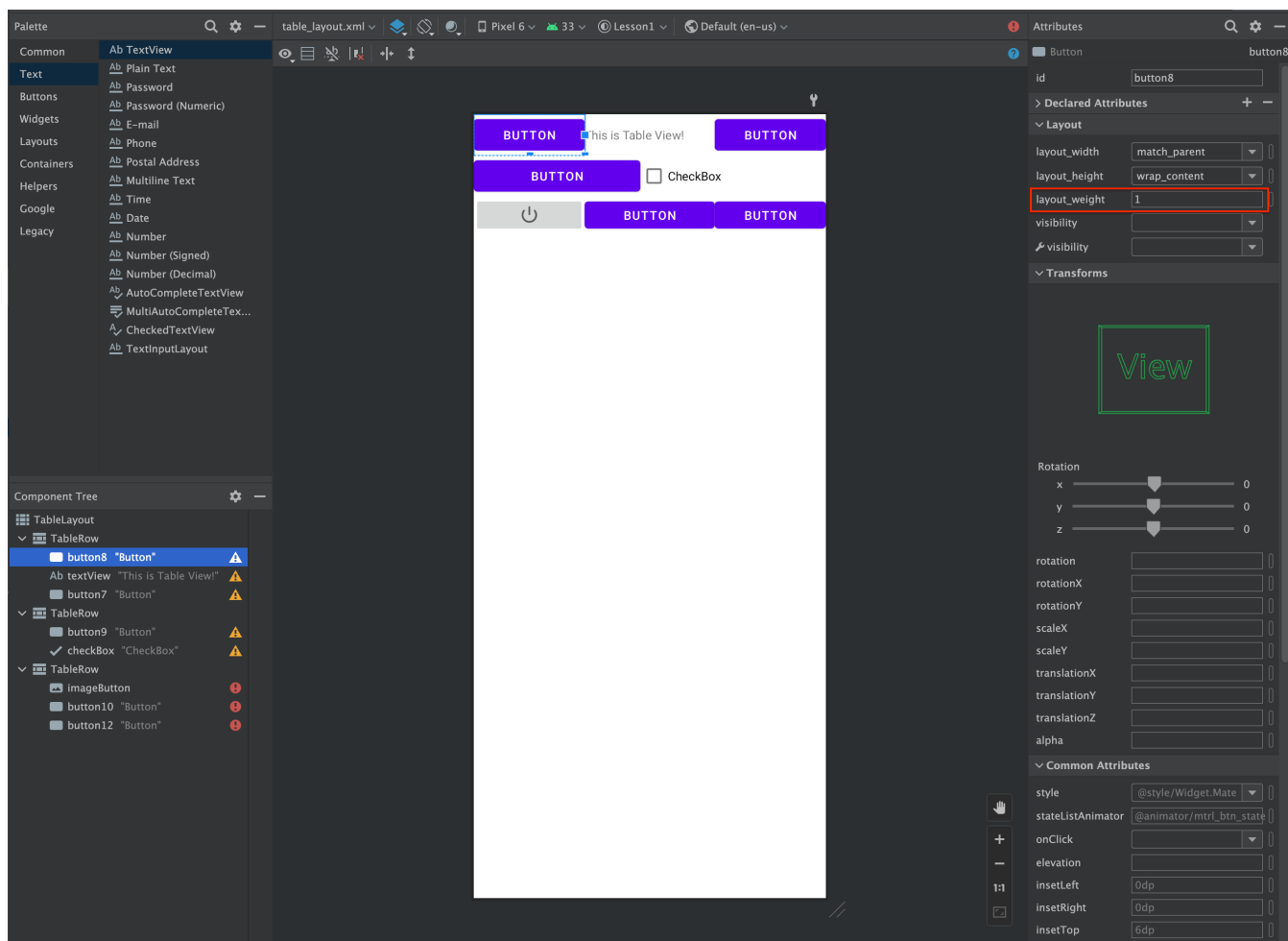


Рисунок 4.3 – Экран TableLayout

Создайте экран приведённый на рис.4.3

4.1.3 ConstraintLayout

Android Studio по умолчанию предлагает использовать «*ConstraintLayout*» при создании разметки экрана. Управление компонентами внутри данного контейнера отличается от предыдущих контейнеров взаимодействия. Требуется добавить элемент «*TextView*» на экран, который отобразится в верхнем левом углу. Далее требуется открыть текстовое представление экрана (вкладка «*Code*»), где элемент «*TextView*» будет подчеркнут красной линией и отображена ошибка:

«This view is not constrained. It only has designtime positions, so it will jump to (0,0) at runtime unless you add the constraints»

Этим сообщением среда разработки сообщает, что «*View*» не привязано к элементам экрана. Для устранения данной ошибки необходимо добавить привязку (*constraints*), позволяющей задавать положение элемента на экране относительно каких-либо других элементов или относительно родительского. Для добавления

привязки «*TextView*» требуется его выделить, после чего отобразится 4 круга по его бокам (рис.4.4). Данные круги используются для создания привязки.

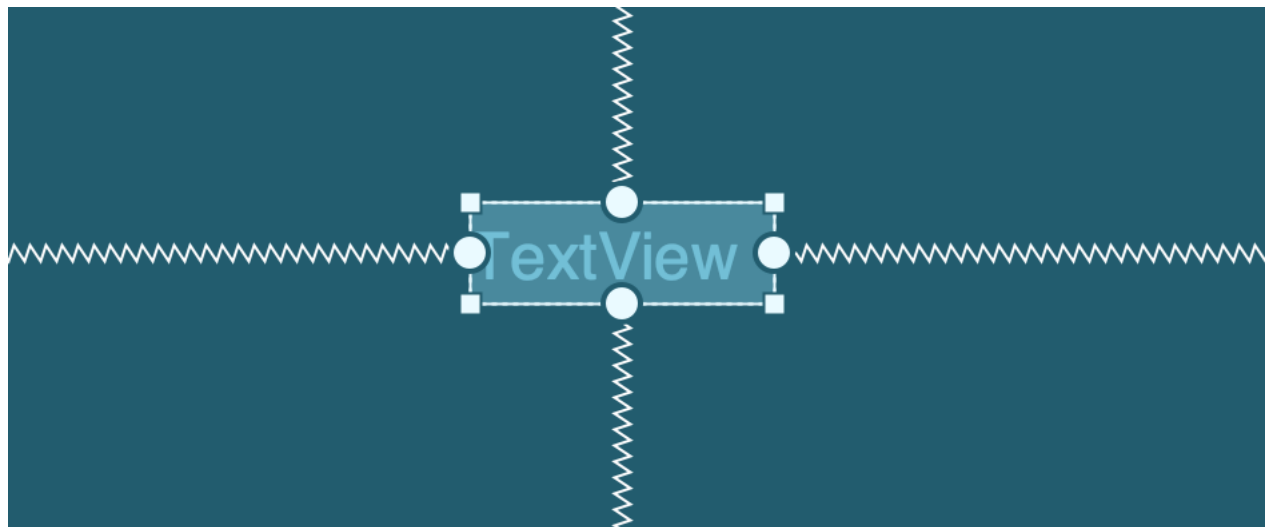


Рисунок 4.5 - Компонент «*TextView*» в контейнере «*ConstraintLayout*»

Элемент «*TextView*» располагается в «*ConstraintLayout*», который в приведенном примере занимает весь экран. Поэтому края «*ConstraintLayout*» совпадают с краями экрана. Создание привязки требует зажима левой кнопкой мыши кружков и перетаскивание его к краям экрана. После привязки возможно перетащить «*TextView*» в требуемое место, чтобы настроить горизонтальный и вертикальный отступы. При перетаскивании отображаются значения отступов. Также имеется возможность привязать «*View*» с обеих сторон по каждой оси. Привязки уравнивают друг друга, и «*View*» будет находится ровно посередине между тем, к чему он привязан слева, и тем, к чему он привязан справа. Т.е. в случае «*View*» находится посередине между левой и правой границами его родителя.

Добавить на экран несколько элементов и привязать их между собой.

4.2 Задание

Создать новый модуль «*control_lesson1*».

Открыть разметку «*activity_main.xml*» (*res>layout*) из меню *palette* перенести следующие элементы и изучить их свойства (область «*attributes*»):

- *Text>textView, plainText (editText)* и т.д.;
- *Buttons>button, imageButton, checkBox* и т.д.;
- *Widgets>imageview* (установить изображение) и т.д..

Требуется создать собственный экран с использованием изученных элементов.
На рисунке 4.5 приведен в качестве примера экран информации о контакте.

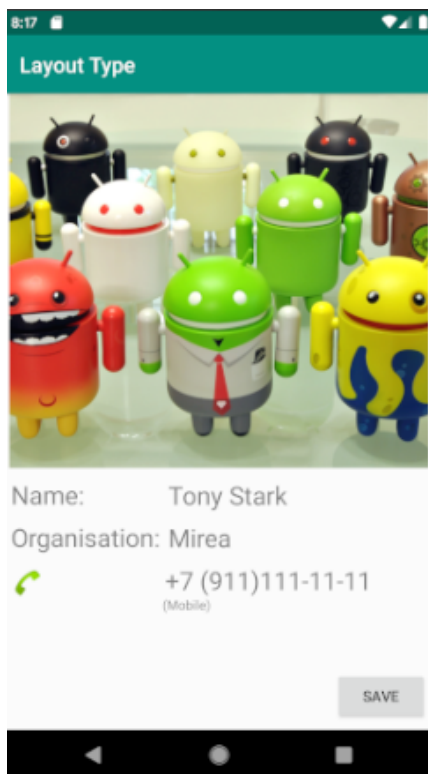


Рисунок 4.6 – Экран задания №1

ГЛАВА 5. LAYOUT-ФАЙЛ В ACTIVITY. СМЕНА ОРИЕНТАЦИИ ЭКРАНА

5.1 Layout-файл

При разработке, каждому «*activity*» сопоставляется одноименный Java-класс (наследник класса *android.app.activity*). При запуске приложения, когда система должна отобразить «*activity*» и предоставить возможность пользователю взаимодействовать с ней, будут вызываться методы связанного класса, определяющих поведение. Далее представлен файл «*MainActivity*»:

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v,
insets) -> {
            Insets systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom);
            return insets;
        }));
    }
}
```

Метод «*onCreate*» вызывается, когда приложение создает и отображает «*activity*». В начале вызывается метод родительского класса через ключевое слово «*super*», выполняющий необходимые процедуры по созданию *activity*. Метод «*setContentView(int)*» связывает код в «*activity*» с разметкой *layout*-файла. В качестве аргумента указывается не путь к *layout*-файлу (*res/layout/activity_main.xml*), а идентификатор файла. Данная константа создается автоматически в файле *r.java*. В данном классе хранятся сгенерированные идентификаторы всех ресурсов проекта (из папки *res/**) для обеспечения возможности взаимодействия с ними. Имена данных идентификаторов-констант совпадают с именами файлов ресурсов (без расширений). Файл «*activity_main.xml*» (*res/layout/*) создается средой разработки вместе с «*activity*» при создании проекта.

5.2 Задание:

Создать *layout*-файл «*activity_second.xml*».

Требуется вызвать контекстное меню директории «res/layout» в модуле и открыть генератор ресурсов: *New> XML> Layout XML File> Layout Resource File*. Для нового файла разметки установить имя «*activity_second.xml*». В созданном layout-файле необходимо добавить на экран элемент «*PlainText*» и изменить его текст на: «*New life for mirea activity!*» и 6 кнопок button.

При создании нового layout-файла «*activity_second*», среда добавит в *r.java* новую константу – «*R.layout.activity_second*». После этого в коде класса *MainActivity* возможно через данную константу указать на новый layout-файл. Для того, чтобы *activity* использовало новый файл «*activity_second.xml*», а не «*activity_main.xml*», требуется открыть «*MainActivity.java*» и изменить аргумент метода *setContentView*:

```
public class MainActivity extends AppCompatActivity {  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        EdgeToEdge.enable(this);  
  
        setContentView(R.layout.activity_main);  
  
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v,  
insets) -> {  
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.system-  
Bars());  
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, sys-  
temBars.bottom);  
            return insets;  
        }));  
    }  
}
```

5.3 Ориентация экрана

Существует два режима отображения приложения на мобильном устройстве: портретный и альбомный. На большинстве телефонов используется по умолчанию портретный режим. Для обеспечения возможности поворота экрана на эмуляторе требуется убедиться, что в меню установлено значение «Auto-rotate» (рисунок 5.1).

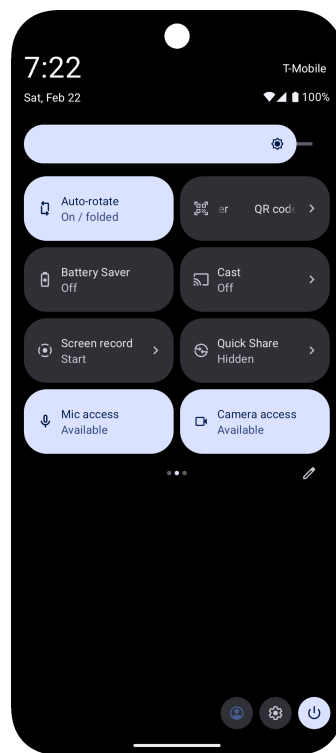


Рисунок 5.1 – Экран задания №1

В качестве примера представлен пример экрана с одним текстовым полем и шестью кнопками. На рисунке 5.2 и 5.3 приведён внешний вид вертикальной и горизонтальной ориентации экрана соответственно. Стоит обратить внимание, что после поворота устройства на 90 градусов пятая и шестая кнопка оказались за пределами видимости.



Рисунок 5.2 – Портретная ориентация экрана



Рисунок 5.3 – Горизонтальная ориентация экрана

С целью исключения данных ситуаций требуется сформировать разметку для другой ориентации. Например, изменить их расположение. В качестве родительского контейнера возможно использовать «*TableLayout*». С его помощью возможно разбить кнопки на две колонки и поместить их в три ряда.

Для данной операции понадобится вызвать контекстное меню у директории «*res*» и выбрать команды «*New | Layout Resource File*» (рис.5.4). В списке «*Available qualifiers*» расположен элемент «*Orientation*», который переносится в правую часть «*Chosen qualifiers*»: с помощью кнопки с двумя стрелками. По умолчанию значение поля «*Directory name*» изменится на «*layout-port*». Т.к. основной целью является поддержка горизонтальной ориентации в выпадающем списке «*Screen orientation*» требуется выбрать «*Landscape*».

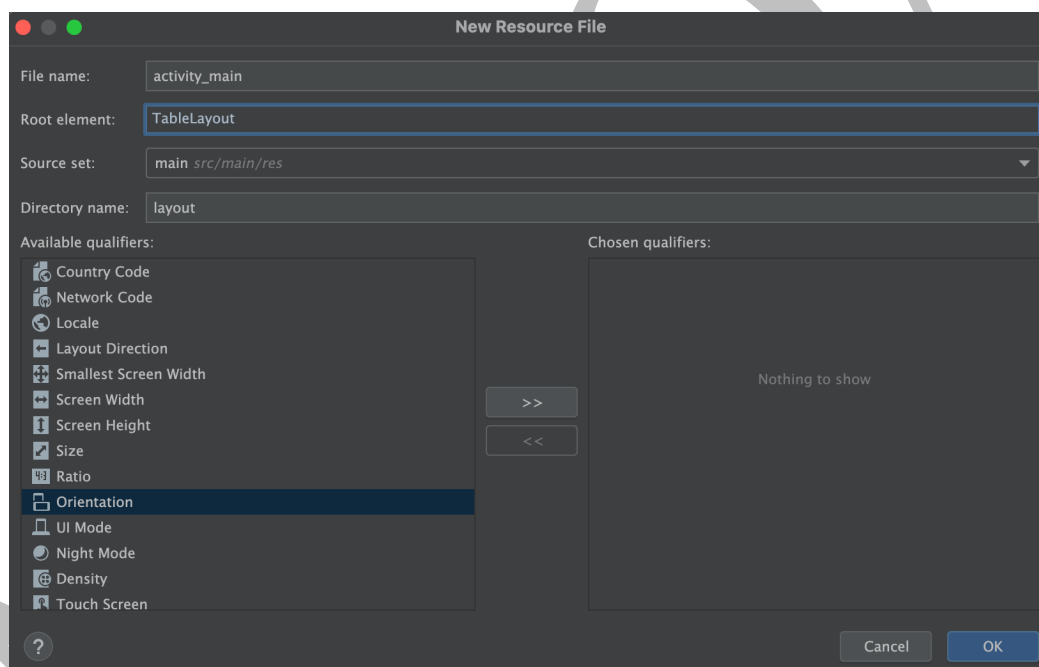


Рисунок 5.2 – Экран создания горизонтальной разметки

После создания альтернативного файла разметки в структуре модуля (вкладка «*android*»), размещено два файла «*activity_main.xml*»: стандартный и *land* (рисунок 5.4).

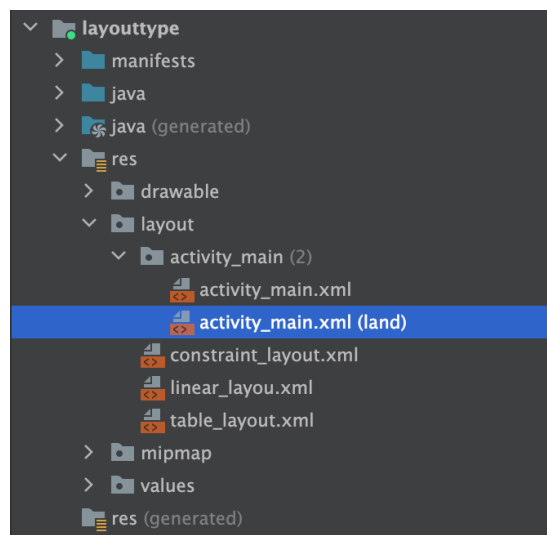


Рисунок 5.3 – Каталог файлов разметки

5.4 Задание

Требуется открыть файл `activity_second.xml (land)` и изменить расположение кнопок так, чтобы все из них были отображены.

Открыть модуль «control_lesson1». Добавить горизонтальную разметку.

ГЛАВА 6. ОБРАЩЕНИЕ ИЗ КОДА К ЭЛЕМЕНТАМ ЭКРАНА. ОБРАБОТЧИКИ СОБЫТИЙ.

Класс «*activity*» представляет собой отдельный экран (страницу) или набор экранов приложения. Приложения могут иметь одну «*activity*», а может и несколько. Каждая отдельная «*activity*» задает отдельное окно для отображения. Стоит отметить, что современный подход создания мобильных приложений заключается в использовании одного «*activity*», но с множеством количеством фрагментов или «*View*».

Обращения к элементу на экране из кода производится через связывания его *идентификатора*. Класс «*R*» содержит идентификаторы всех ресурсов, расположенных в каталоге «*res*». Для каждого типа ресурсов в классе «*R*» создается внутренний класс (например, для всех графических ресурсов из каталога «*res/drawable*» создается класс *R.drawable*) и для каждого ресурса данного типа присваивается идентификатор.

В файле *activity_main.xml* для элемента «*TextView*» требуется указать:

```
TextView tv = findViewById(R.id.textviewStudent);
```

Обращение к текстовому полю программным способом осуществляется с помощью вызова метода «*findViewById*», возвращающий по идентификатору ссылку на «*View*» объект. Требуется добавить в класс «*MainActivity.java*» после метода связывания файла разметки «*setContentview*» создание объекта «*TextView*»:

```
TextView myTextView = (TextView) findViewById(R.id.textview);
```

В случаях, когда «*View*» подчеркнуто красной линией скорее всего данный класс не добавлен в секцию «*import*». Нажмите «*CTRL+SHIFT+O*» для автоматического обновления импорта.

Переменная «*myTextView*» является типом данных «*TextView*», а результат метода «*findViewById*» приводится из «*View*» к «*TextView*». После успешной инициализации возможно применять к «*myTextView*» методы класса «*TextView*». Для примера используется метод *setText*:

```
myTextView.setText("New text in MIREA");
```

Далее требуется скомпилировать и запустить проект для отображения нового текста в поле. Взаимодействие с кнопками осуществляется аналогичным способом. В файл разметки требуется добавить кнопку «*Button*» и установить идентификатор «*id=@+id/button*», а в классе «*MainActivity*» создать переменную типа «*Button*»:

```
Button button = findViewById(R.id.button);
```

Вызов метода «*setText*» с аргументом изменит значение текста кнопки:

```
button.setText("MireaButton");
```

Элемент «*CheckBox*» также требует установки идентификатора «*id=@+id/check_box*». Для установки метки используется метод «*setChecked*», меняющий параметр на значение «*Checked*».

```
CheckBox checkBox = findViewById(R.id.checkBox);  
checkBox.setChecked(true);
```

В результате, метод «*findViewById*» используется, чтобы по идентификатору получить ссылку на объект, соответствующей какому-либо «*View*»-элементу и далее выполнять вызов необходимых методов объектов.

ГЛАВА 7. ОБРАБОТЧИКИ СОБЫТИЙ

Элемент «Button» требует наличие обработчика (слушатель – «*listener*»), который присваивается с помощью метода «*setOnClickListener*». Когда на кнопку нажимают, обработчик реагирует и выполняет код из метода «*onClick*» (рис.7.1).

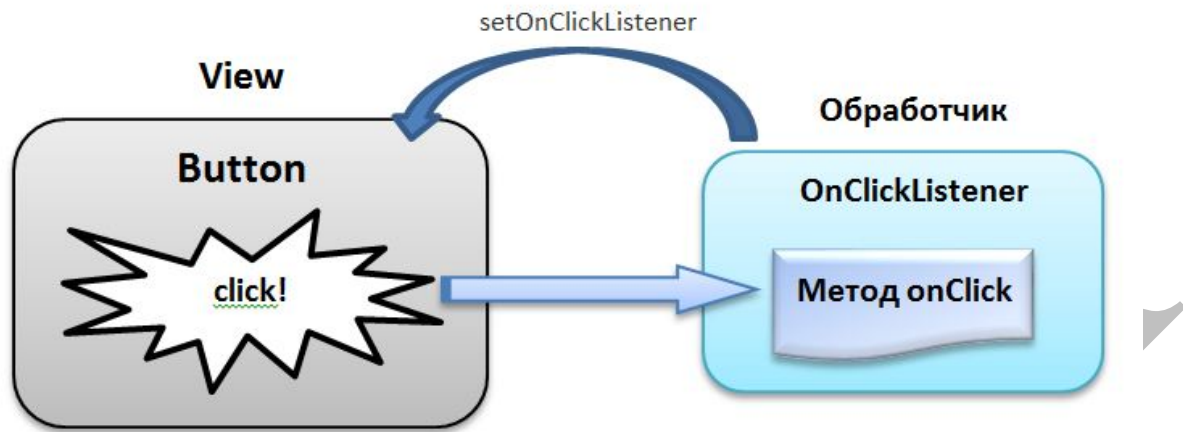


Рисунок 7.1 - Принцип обработки событий

Соответственно для реализации обработки событий на кнопке необходимо выполнить следующие шаги:

- создать обработчик событий;
- реализовать метод «*onClick*»;
- установить обработчик для кнопки.

Требуется создать новый модуль. В меню «*File>New>New Module>Phone & Tablet Module>Empty Activity*». Название проекта «*ButtonClicker*».

Требуется открыть разметку «*activity_main.xml*» и добавить «*TextView*» с идентификатором «*id="@+id/tvOut"*» и две кнопки: «*WhoAmI*» (*id="@+id/btnWhoAmI"*) и «*ItIsNotMe*» (*id="@+id/btnItIsNotMe"*). Внешний вид экрана представлен на рисунке 7.2.

Требуется, чтобы по нажатию кнопки менялось содержимое «*TextView*». По нажатию кнопки «*WhoAmI*» – выводится текст: «*Мой номер по списку № X (по журналу)*», по нажатию «*ItIsNotMe*» – «*Это не я сделал*».

Данный экран связан с классом «*MainActivity.java*». Объявление объектов требуется вынести за пределы метода «*onCreate*» и сделать глобальными

переменными. В методе «onCreate» производится инициализация объектов с помощью метода «*findViewById*».



Рисунок 7.2 – Экран приложения

В итоге должен получиться следующий код (за):

```
public class MainActivity extends AppCompatActivity {  
    private TextView textViewStudent;  
    private Button btnWhoAmI;  
    private Button btnItIsNotMe ;  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
        textViewStudent = findViewById(R.id.textViewStudent);  
        btnWhoAmI = findViewById(R.id.btnWhoAmI);  
        btnItIsNotMe = findViewById(R.id.btnItIsNotMe);  
    }  
}
```

Инициализированные объекты соответствуют «*View*»-элементам экрана. В первую очередь требуется реализация нажатий на кнопки. Для решение данной задачи существует метод «*setOnClickListener (View.OnClickListener l)*». Аргументом метода является объект с интерфейсом «*View.OnClickListener*». Именно этому

объекту кнопка делегирует обработку нажатия. Требуется создать данный объект в методе «*onCreate*»:

```
protected void onCreate(Bundle savedInstanceState) {  
    ...  
    View.OnClickListener oclBtnWhoAmI = new View.OnClickListener() {  
        @Override  
        public void onClick(View v) {  
            textViewStudent.setText("Мой номер по списку № X");  
        }  
    };  
}
```

Для добавления зависимостей в класс требуется навести курсор на подчёркнутое слово и добавить недостающие элементы (*Alt+Enter*) «*View.OnClickListener*». Таким образом, создаётся объект «*oclBtnWhoAmI*», который реализует интерфейс «*View.OnClickListener*». Реализация логики обработки нажатия выполняется внутри метода «*onClick*». Далее требуется передать обработчик нажатия кнопке с помощью метода «*setOnClickListener*».

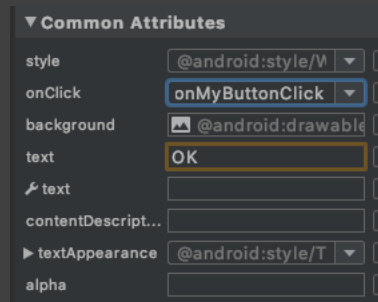
```
protected void onCreate(Bundle savedInstanceState) {  
    ...  
    btnWhoAmI.setOnClickListener(oclBtnWhoAmI);  
}
```

В результате должен получиться следующий код:

```
public class MainActivity extends AppCompatActivity {  
    private TextView textViewStudent;  
    private Button btnWhoAmI;  
    private Button btnItIsNotMe ;  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
        textViewStudent = findViewById(R.id.textViewStudent);  
        btnWhoAmI = findViewById(R.id.btnWhoAmI);  
        btnItIsNotMe = findViewById(R.id.btnItIsNotMe);  
        View.OnClickListener oclBtnWhoAmI = new View.OnClickListener() {  
            @Override  
            public void onClick(View v) {  
                textViewStudent.setText("Мой номер по списку № X");  
            }  
        };  
        btnWhoAmI.setOnClickListener(oclBtnWhoAmI);  
    }  
}
```

Также существует альтернативный способ обработки событий нажатий - атрибут «*onClick*» в xml (на панели свойств отображается как On Click):

android:onClick="**onMyButtonClick**"



В классе требуется создать метод:

```
public void onMyButtonClick(View view)
{
    // выводим сообщение
    Toast.makeText(this, "Ещё один способ!", Toast.LENGTH_SHORT).show();
}
```

Задание:

Создайте обработчик события для кнопки «*btnItIsNotMe*» вторым способом. Добавьте элемент «*CheckBox*» изменяющий свое состояние при нажатии на кнопки вместе «*TextView*».